

Description of the Surface Water Accounting Model Program (SWAc)

Groundwater Science Ltd. July 2025 (version 2.0.0)

Contents

1	Licence	1
2	Warranty and Liability.....	1
3	Acknowledgements.....	1
4	Purpose	2
5	Represented Processes and Program Workflow	3
5.1	Main model processes.....	5
5.2	Anthropogenic processes	5
5.3	Attenuation processes.....	5
5.4	Special use cases.....	5
5.5	Introductory note on lateral flow mechanisms.....	6
6	Detailed flow mechanism description	6
6.1	Canopy interception	6
6.2	Snow melt.....	7
6.3	Rapid runoff.....	7
6.4	Runoff recharge with downstream routing.....	8
6.5	Single cell runoff recharge without downstream routing	9
6.6	Macropore recharge.....	9
6.7	Soil moisture accounting and actual evapotranspiration (AE) calculation.....	10
6.8	Percolation rejection	12
6.9	Leakage below the root zone	12
6.10	Interflow and combined near surface lateral flow processes	12
6.11	Attenuation of recharge as it passes through the unsaturated zone	13
6.12	Solute transport.....	13
6.13	Surface water attenuation.....	15
6.14	Surface water ponding	15
6.15	Surface water routing.....	16
6.16	Surface water discharge	16
6.17	Surface water abstraction	16
7	Model master control file and input methods.....	17
7.1	Input file structure.....	17
7.2	Global inputs.....	17
7.3	Spatially defined static properties.....	21
7.4	Climatic time series	22
7.5	Main model processes.....	24
7.5.1	Canopy interception.....	24

7.5.2	Snow melt.....	26
7.5.3	Rapid runoff.....	33
7.5.4	Runoff recharge with downstream routing	36
7.5.5	Single cell runoff recharge without downstream routing.....	39
7.5.6	Macropore recharge	39
7.5.7	Soil moisture accounting and actual evapotranspiration calculation.....	42
7.5.8	Percolation rejection.....	43
7.5.9	Leakage below the root zone	44
7.5.10	Interflow	47
7.5.11	Attenuation of recharge	50
7.5.12	Solute transport.....	51
7.5.13	Surface water release	56
7.5.14	Surface water ponding.....	57
7.5.15	Surface water routing	60
7.5.16	Surface water discharge	64
7.5.17	Surface water abstraction.....	65
8	Model output files.....	66
8.1	MODFLOW output files	66
8.2	Breakdown output files	66
9	Code overview and Runtime	68
9.1	Node-wise parallelisation	68
9.2	NumPy vectorisation	68
9.3	Cython integration.....	69
10	Usage.....	69
11	References	70
Appendix A.	Template master control file	71
Appendix B.	Description of Input and Output.....	87

1 Licence

GNU General Public Licence v3.0

Version 3, 29 June 2007

Permissions of this strong copyleft licence are conditioned on making available complete source code of licensed works and modifications, which include larger works using a licensed work, under the same license. Copyright and license notices must be preserved. Contributors provide an express grant of patent rights.

Full Terms and Conditions of the GNU-GPL Licence can be found here: <http://www.gnu.org/licenses/>

2 Warranty and Liability

There is no warranty for the program. Groundwater Science Ltd. provides the program in the hope that it will be useful. It is supplied “as is” without warranty of any kind, either expressed or implied, including, but not limited to the implied warranties of merchantability and fitness for a particular purpose. The entire risk as to the quality and performance of the program is with you. Should the program prove defective, you assume the cost of all necessary refinement or correction.

In no event will Groundwater Science Ltd., or any other party who modifies and/or conveys the program as permitted in GNU-GPL3, be liable to you for damages, including: general, special, incidental or consequential damages arising out of the use or inability to use the program.

3 Acknowledgements

The Surface Water Accounting Model (SWAc) is developed by Groundwater Science Ltd. The program however collates a large body of work developed by the wider water management community over the last two decades. The program has benefitted from the developments in similar software including, and not limited to: the UK EA recharge code, PRMS (Markstrom, et al., 2015) and RRRR (Heathcote, et al., 2004). The program implements similar literature reviewed component processes as previously collated into these programs.

SRK Consulting UK provided support and review in the early development of the multi-environment aspect of SWAc, datasets for testing and report review.

Mott MacDonald supported the development of runtime efficiency through parallelisation and process output validation.

Notable, individual contributors are listed below:

- Marco Lagi of the New England Earth Systems Institute who greatly aided the architecture of the program;
- Chris Foley of Nice Code Ltd, along with Alastair Black, Mal Graham and Charalampos Ntiggakis of Groundwater Science Ltd. for ongoing maintenance and updates.
- Will Witterick (formerly of Groundwater Science Ltd) for past maintenance and updates.
- Tim Lewis and Rob Soley for the substantial contributors to UK runoff-recharge modelling.
- Professor Ken Rushton and Jan van Wonderen who authored the majority of the underlying component processes and Dave Johnson, Mark Whiteman, Paul Hulme and Sarah Alloisio who

undertook the original collation and review of literature processes relevant to runoff-recharge accounting.

4 Purpose

SWAc (Surface Water Accounting Model) is a program that can be used in isolation for catchment hydrology and water balance analysis, or in combination with MODFLOW, for holistic hydrological-hydrogeological studies. The original motivation for SWAc development was to make available a modular tool that could assist hydrogeological assessments in Equatorial, Tropical, Temperate and Arctic climatic zones for the mining and water resource industries. In addition, it was required to be fast, permit changing ground conditions and make use of freely available global datasets.

SWAc is a fully distributed model, mesh style agnostic and supports variable cell sizes for scale dependant simulations. It is designed for use with Unstructured Mesh models, for example, MODFLOW 6 (Langevin, et al., 2017), MODFLOW USG (Panday, et al., 2013) and can be used in legacy structured variants like MODFLOW 1996 (Harbaugh & McDonald, 1996).

The program utilises surface and near-surface spatially distributed properties and timeseries climatic inputs. Spatially distributed input parameters can be varied through the simulation: rainfall, potential evapotranspiration, crop life cycles, pipe network leakage and land-use change are common examples of transient, spatially distributed inputs. The model partitions water through the simulated system to output MODFLOW compatible recharge to groundwater, surface flows and/or potential riparian fluxes. SWAc optionally outputs: Recharge (RCH), Evapotranspiration (EVT) and Segmented Evapotranspiration (ETS) packages for integration with MODFLOW groundwater models.

Surface water drainage networks can be optionally included as topologically routed watercourses for robust groundwater-surface water interaction simulation for perennial and ephemeral systems. For this application SWAc produces MODFLOW Stream (STR) or MODFLOW Stream Flow Routing (SFR) packages for integration with MODFLOW.

Additional non-MODFLOW outputs include: detailed water balance breakdowns across user specified areas of interest; and, spatial distributions of flows through each modelled process at specified dates.

An overview of the processes represented is included in section 5.

A challenge in the application of fully distributed runoff-recharge models is data availability. Particularly the spatial distribution of surface conditions, lithology and land use. Recently available high-resolution satellite and radar datasets are of great benefit, typically providing inputs with global coverage and at 50m or finer spatial resolution. Such data can be used to augment ground-based datasets or in areas where no other data exists. The SWAc program is designed to ease the implementation of these datasets to improve the spatial representation of, for example, land cover, snow, surface moisture and vegetation change over time.

Runtime is a major consideration in SWAc. Various strategies to minimise runtime have been employed to ease calibration and encourage stochastic and uncertainty analysis applications. The program can be run on cloud platforms and makes use of multi-core computer architectures. In addition, the YAML language system can be optionally utilised for model inputs which eases SWAc linkage to other software (e.g. PEST or Model Interfaces)

The program is designed to be open source, free to use and adapt. It is released under the GNU-GPLv3 open-source licence. The structure of the program is modular, well commented and written in the popular Python language in order to facilitate broad community inclusion.

Above all the desire is that the program will be used, and be useful. We hope that it meets your needs and critical comments and suggestions for enhancements are welcome. We also invite you to develop additional or alternative modules. The program is released freely. It is therefore asked that all developments are uploaded to the central GitHub repository (which can be found at <https://github.com/GWSci/SWAc>) and made available to the community freely.

5 Represented Processes and Program Workflow

A SWAc model is built as a collection of polygons which cover the two-dimensional surface being simulated; each polygon has an associated vertical workflow representing the flow mechanisms water may pass through between surface and groundwater. When used in conjunction with MODFLOW the surface polygon 'mesh' can be mapped to the MODFLOW model unstructured mesh or structured grid. This results in simple mapping of nodes 1 to n in SWAc to 1 to n in MODFLOW, as shown in Figure 1.

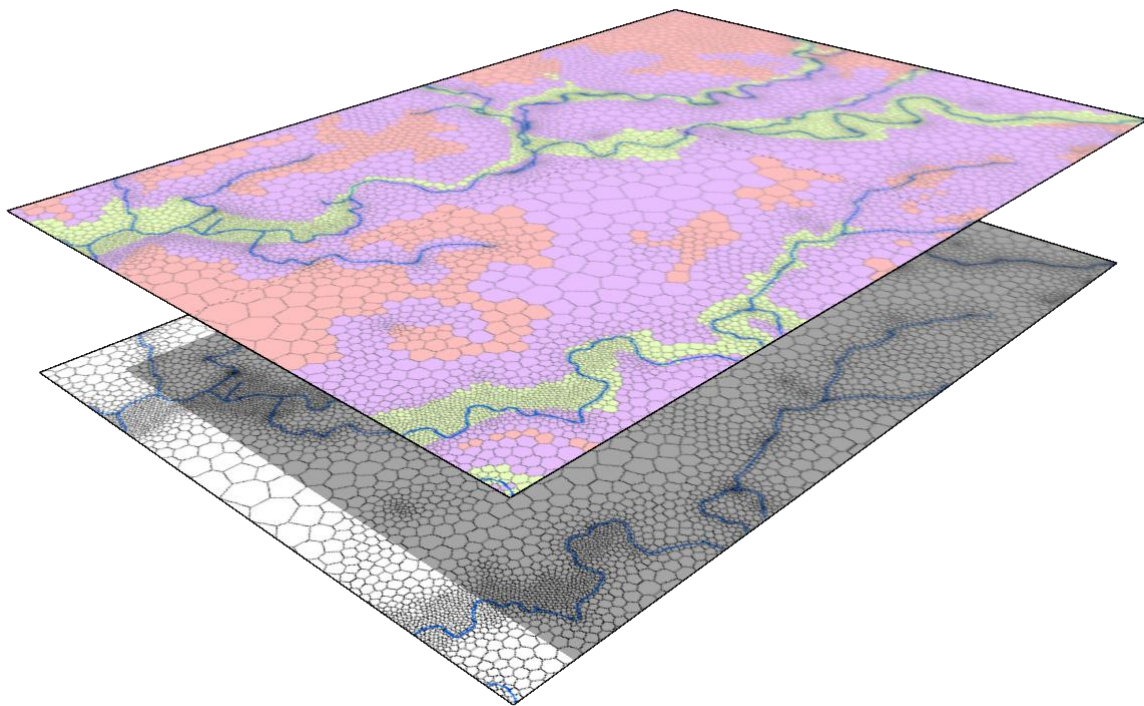


Figure 5.1 Correspondence of SWAc and MODFLOW model mesh

The vertical workflow is summarised in figure 2. All current model processes are shown; arrows denote the order in which each process is calculated relative the next.

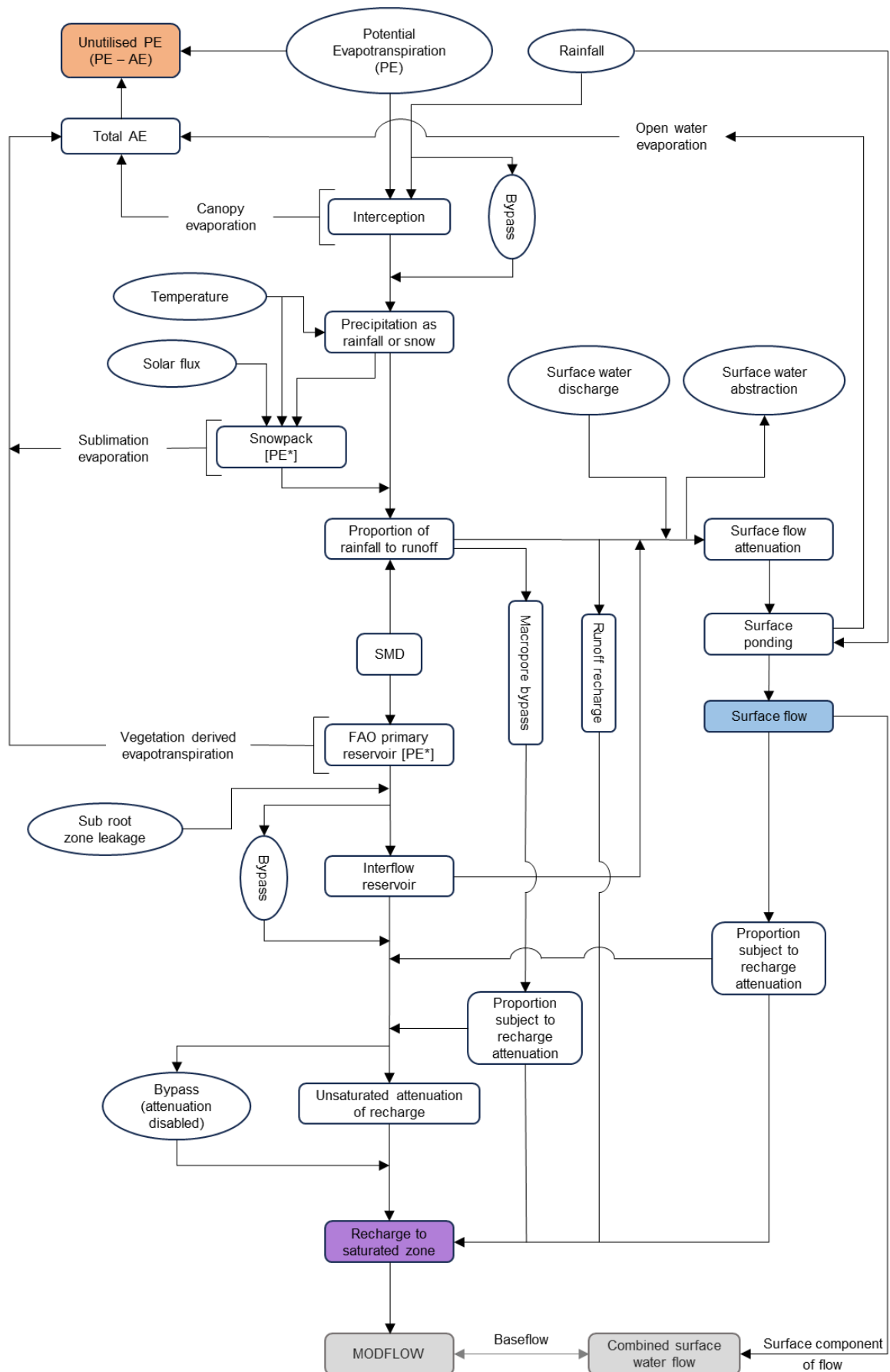


Figure 5.2 Workflow of SWAc Processes

The majority of flow mechanisms in the SWAc vertical workflow are derived from existing studies referenced in literature, many of these are also included in the UK EA Recharge code, RRRR (Heathcote, et al., 2004) and PRMS (Markstrom, et al., 2015). Processes in SWAc can be optionally enabled or disabled depending on the needs of the site being modelled. Detailed flow mechanisms and the order of calculations for each process are discussed in more detail in section 6.

The workflow moves from ground surface down through the soil zone and unsaturated zones. The model requires, as a minimum, timeseries rainfall data, but normally also timeseries potential evapotranspiration (PE). Snow pack build-up and melt processes additionally require temperature and potentially solar flux time series data. Each flow mechanism requires parameterisation to control the action of that process. These are input either explicitly - defined for each spatial cell, or, zone numbers are assigned to cells and properties linked to zone numbers. The method of property mapping is discussed in section 7.3.

Model processes that can be enabled/disabled include the following. Each model process is identified in SWAc by means of a keyword, the keyword for each process is mentioned in brackets for each process. Where possible, mechanisms are described in chronological order of calculation, anthropogenic and attenuation type processes are listed at the end.

5.1 Main model processes

- Canopy or vegetation interception and evaporation ('canopy')
- Temperature dependant precipitation as snowfall or rainfall ('snow_process_simple')
- Snowpack development and melt ('snow_process_complex')
- Partitioning of rainfall/melt water as rapid runoff ('rapid_runoff')
- Runoff recharge with downstream routing ('swrecharge')
- Single cell runoff recharge without downstream routing ('single_cell_swrecharge')
- Rapid recharge bypassing the soil store such as fracture derived recharge ('macropore')
- Soil Moisture Accounting and actual evapotranspiration (AE) calculation ('fao')
- Rejected percolation from the soil moisture zone ('percolation_rejection')
- Additional shallow ponding enhancements to the Soil Moisture Accounting process;
- Urban water mains, sewer, canal leakage, essentially any leakage occurring below the root zone ('subroot_leakage')
- Interflow and combined near surface lateral flow processes ('interflow')

5.2 Anthropogenic processes

- Discharge to surface water, such as sewage treatment plants ('swdis')
- Abstraction from surface water, such as water utility treatment plants ('swabs')

5.3 Attenuation processes

- Attenuation of surface water to account for time of concentration in larger catchments ('sw')
- Attenuation of recharge as it passes through the unsaturated zone ('recharge_attenuation')

5.4 Special use cases

- Unsaturated zone drip release

- Attenuation of surface water (leaky dams method)
- Advanced recharge attenuation (currently outside SWAc)
- Unused PE passing to MF .evt file

5.5 Introductory note on lateral flow mechanisms

In most flow mechanisms the calculations in a given cell are made independently of neighbouring cells. Lateral connection can however be applied between vertical workflows to permit the accretion and depletion of river flows as water passes down through the surface drainage system. In such cases vertical workflows (Figure 2) can be linked by a topologically routed surface flow network with each 'surface flow' output linked to input into the downstream cell. This mechanism facilitates the ability of processes such as "runoff recharge" to consider the collective volume of all upstream cells as opposed to just the local cell.

Conceptual settings which include surface runoff flow concentration on weakly permeable material passing onto lithologies which permit high infiltration can be represented resulting in enhanced infiltration on, and downstream of, the contact. Routed and accumulated surface water is commonly an input to groundwater models in areas of potential groundwater-surface water interaction and ephemeral conditions. The accreted flow volumes are processed into MODFLOW compatible 'Stream' (STR) or 'Surface Flow Routing' (SFR) files.

6 Detailed flow mechanism description

6.1 Canopy interception

In woodland areas, the canopy interception process can be enabled to simulate canopy storage and actual evapotranspiration prior to precipitation reaching ground level. Two input parameters are required, namely a storage limit (mm), and a throughfall fraction. Unlike the soil moisture store, canopy interception does not allow for storage persisting across model-steps, store gets fully depleted at the end of each time-step. Conceptually, this equates to a short-term store that collects a portion of rainfall that becomes converted to actual evapotranspiration.

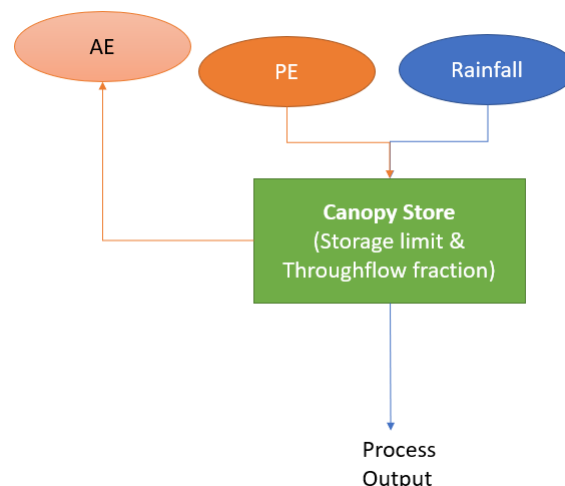


Figure 6.1 Schematic representation of the canopy interception process

The calculation of canopy storage takes into account three possible scenarios as follows:

1. If Throughflow amount > Store limit, THEN AE = Store Limit

2. *If Throughflow amount < Store limit, THEN AE = Throughflow amount*
3. *If either Throughflow amount OR Store limit > Vegetation factored PE, THEN AE = Vegetation factored PE*

In this way the process is always limited by vegetation factored PE.

Note: the amount of subsequent AE that can occur from the soil moisture store is limited by this process because only the remainder of vegetation factored PE is applied to the soil store.

6.2 Snow melt

The snow process with SWAc consists of two different sub-process: (i) the simple snow melt process, and (ii) the complex snow melt process. Either of these processes can be enabled or disabled.

The simple snow melt process controls the partitioning of precipitation between rainfall and snowfall, depending on a set of user defined temperature thresholds. Three user-specified parameters are required for this process, namely the starting snowpack (in mm), an upper temperature threshold for snowfall (in degrees C), and a lower temperature threshold for snowfall (in degrees C). The process will compare the temperature specified in the temperature timeseries for each time period to the user specified thresholds and partition the precipitation between rainfall and snowfall. Similarly, it will estimate snow pack build-up or snow melt for each time period.

The complex snow melt process is based on the SnowMelt calculator of the EcoHydRology package. This package was originally developed in R and distributed as part of the CRAN repository. The SnowMelt function was adapted and rewritten in Python to be incorporated into the SWAc main model processes. Snow melt is estimated using an energy balance approach originally discussed by Walter et al. (2005). This approach uses local hydrological parameters including minimum and maximum daily temperature, precipitation, and windspeed. Furthermore, it requires topological parameters including slope and aspect of the terrain, ground albedo, tree cover, surface emissivity ration, as well as location latitude for solar radiation estimations.

6.3 Rapid runoff

Rapid runoff is the surface flow that occurs following rainfall events. The typical end-point for the runoff water would be the local streams and rivers. The water subject to runoff is the sum of rainfall input which has passed through or bypassed the canopy process and snow melt. In areas of heterogeneous surface infiltration characteristics runoff potentially could percolate to the groundwater in high infiltration regions downstream of where the runoff was originally generated. This downstream pathway to groundwater mechanism is simulated as 'runoff recharge' and calculated independently of 'rapid runoff'.

The rapid runoff mechanism employed is dependent on both: daily rainfall flux; and, antecedent soil moisture conditions. The methodology applied was first described in TopModel (Beven & Kirkby, 1979) and has been additionally applied in a host of studies and programs including Bishop and Rushton (1993), Heathcote et al. (2004), and is further discussed in Rushton (2003).

The antecedent soil moisture conditions (SMD) is calculated in the Soil Moisture process (Section 6.7). If the Soil Moisture process is disabled the SMD utilised is 0, i.e. fully saturated. Soil moisture and rainfall intensity classes are defined by the user depending on the level of complexity required. There is, however, no real restriction of classes that can be assigned. Refer to Section 7 for details on user defined rainfall-SMD-runoff tables.

Note that differing runoff coefficient tables can be assigned to different spatial zones. Spatial zones can be delineated based on topographic slope, land-use, soil type, or any other pertinent spatial factor

or combination of factors that can be mapped to the model mesh. As a general guideline, more rainfall classes would usually be applied to an arable zone compared to an urban zone although different use-cases may warrant alternative setups.

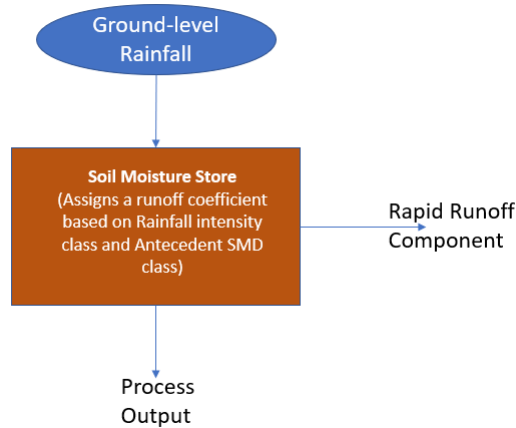


Figure 6.2 Schematic representation of the rapid runoff process

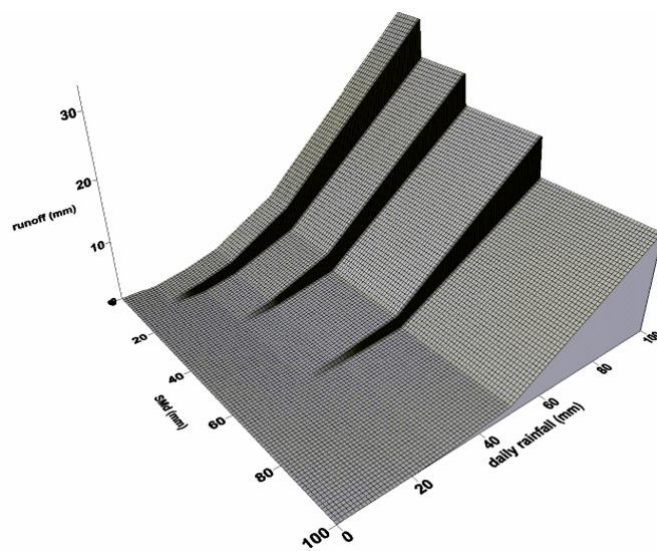


Figure 6.3 Illustrative example of runoff variation with rainfall and SMD class

The calculation of rapid runoff proceeds as follows on a per cell basis:

1. *Lookup of the applicable rainfall intensity class (user defined) and antecedent SMD class (user defined) to return a single runoff coefficient*
2. *Multiply the runoff coefficient by the ground level precipitation to get the rapid runoff value*

It is worth noting that only a single rainfall intensity class and a single SMD class are used per day, and so a single runoff coefficient is applied to all rainfall occurring within that day.

6.4 Runoff recharge with downstream routing

Runoff recharge is a similar process to macropore recharge (see Section 6.1.6), although it acts on the water within the surface water flow network as opposed the water entering the cell from rain and snow melt. The typical application is to represent recharge concentration along contacts between weakly permeable and high infiltration capacity surface lithologies. Runoff is generated within weakly

permeable areas and flows down through the topologically routed surface water network. Flow from upstream cells is passed to downstream neighbours. When surface conditions change to high infiltration materials the concentrated surface flow volumes can substantially enhance local recharge resulting in heterogeneous recharge along permeable contacts.

The variables to control the process are identical to the macropore process. The primary difference however is the input to the process is the surface water network flow volume and includes the net flow from all upstream cells.

6.5 Single cell runoff recharge without downstream routing

Single cell runoff recharge functions in a similar way to its routed counterpart, although all water apportioned to this process originates as rainfall in the same cell. This simplification allows substantially faster run times in models where there are large numbers of cells subject to runoff recharge, or where cells receiving runoff recharge would otherwise have large catchments.

This process is a legacy feature and is no longer maintained.

6.6 Macropore recharge

Immediate groundwater level response to recharge events is commonly observed in fractured, bioturbated and/or solutionally enhanced environments. Flow through rapid pathways directly to groundwater are simulated with the macropore recharge mechanism.

Macropore represents a user-defined proportion of the available water at the ground surface that bypasses the standard infiltration pathway (the runoff, soil moisture accounting and interflow processes). It can be turned on and off using an activation and limit threshold. The activation term turns on macropore bypass when the amount of available water at surface exceeds this value. The product of the flux exceeding the activation limit and the user-defined proportion up to the limit threshold is assigned to macropore.

The activation threshold for macropore can be set to either rainfall intensity or SMD. If set to rainfall intensity, then a relatively greater volume of macropore recharge can be expected; as soon as rainfall intensity of a particular day exceeds the activation limit then the macropore pathway is activated. Relatively less macropore recharge will occur if the activation threshold is set to SMD. In this case, the build-up of soil moisture generally occurs over a longer time in response to prolonged rainfall, and therefore the macropore pathway takes a longer time to become activated. Either case may be applicable according to the conceptual understanding of your study area.

Macropore recharge can be permitted to proceed directly to the output combined recharge for the cell, or first be subject to recharge attenuation, or a proportion of each. A final proportion variable is used to specify if it is 100% direct, 0% (all subject to attenuation) or something in between respectively.

To summarise, five user defined input parameters are required for this process, namely:

- An activation option to be based on rainfall intensity or SMD (macropore_activation_option)
- An activation level in mm above which macropore can occur (macropore_activation)
- The proportion of available water assigned to macropore (macropore_proportion)
- A limiting level in mm above which no further macropore can occur (macropore_limit)

- The macropore attenuation proportion specifying direct or attenuated macropore (macropore_recharge)

The calculation of rapid runoff proceeds as follows on a per cell basis:

1. *Lookup the available water (either as rainfall or soil moisture) exceeding the macropore activation level*
2. *Multiply the macropore proportion term by the amount of water exceeding the activation level*
3. *Ensure the macropore value does not exceed the limit term, if it does then the limit value is returned*
4. *The available macropore value is assigned to direct recharge and attenuated recharge based on the attenuation proportion*

Proportions and limits can be set to zero in areas where macropore recharge is not required.

6.7 Soil moisture accounting and actual evapotranspiration (AE) calculation

The Soil Moisture Accounting process calculates the proportion of incoming water which replenishes soil moisture, the flux of actual evapotranspiration realised, and the water which continues to percolate downwards past the root zone. On days of no rainfall input actual evapotranspiration and soil moisture deficit increases are calculated.

The Soil Moisture-AE-Percolation mechanism applied is the FAO method (Allen, et al., 1998). This method has been widely adopted internationally and does not have onerous input parameter requirements. Substantial reference values are available in the FAO associated literature.

The process is optionally enhanced in SWAc by the inclusion of a shallow soil store to permit additional actual evapotranspiration on dry days from stored or ponded shallow water from previous rainfall events.

A further augmentation has been added to limit the through-flow of percolating water in extreme water surplus events. If this functionality is enabled a mm/d flux limit is imposed, exceedances are rejected and directed towards rapid runoff.

Water entering the soil moisture mechanism is the remainder of rainfall that has been subject to the enabled preceding processes, including canopy interception, snow melt, macropore bypass and rapid runoff. Such input usually occurs on days with rainfall events or snow melt.

The Potential Evapotranspiration input to the model is typically based on the FAO-Penman Monteith equation. This simplified form of the Penman Monteith equation provides a potential evapotranspiration rate for a hypothetical grass reference crop with a sufficient supply of water to avoid water stress. The magnitude of the available PE is altered from the reference crop value to the appropriate vegetation type by the K_c factor (crop coefficient). This is spatially distributed by vegetation type (land-use). The uplift is typically steady state, however can have a transient implementation to represent vegetation change over time or explicit crop rotation.

The evaporative demand from the soil zone is initially met from the input water to the process. Additional demand is met by the soil-root zone imparting an increased Soil Moisture Deficit. In events where water inputs exceed the evaporative demand the Soil Moisture Deficit is reduced.

The magnitude of Potential Evaporation manifesting as Actual Evapotranspiration depends on the water stress coefficient k_s , which is a function of the Soil Moisture Deficit relative to the thresholds of Readily Available and Evaporable Water (RAWREW) and Total Available and Evaporable Water (TAWTEW), Figure 6.4. The AE is the product of k_s and PE, (where PE represents the k_c factored rate as described above). With a Soil Moisture Deficit lower than RAWREW (wet root zone conditions), evapotranspiration is simulated at the k_c factored potential rate (and hence $k_s = 1.0$). With a Soil

Moisture Deficit greater than TAWTEW (dry conditions) no Actual Evapotranspiration is realised (and hence $k_s = 0.0$). The TAWTEW is analogous to the plant wilting point. The region between these thresholds has a linear sliding scale of k_s . This differs from formerly common approaches (for example Penman and Grindley) which utilised a fixed 0.1 value for the equivalent parameter within the moderately stressed region. The implementation of FAO applied is described more fully in Hulme et al. (2001).

RAWREW and TAWTEW values can be input directly, or alternatively calculated externally from wilting point (WP), field capacity (FC), rooting depth (Zr), and depletion fraction (P) data prior to inputting into SWAc. Substantial reference data exist for vegetation characteristics and suggested values are available for soil textures where no local field analysis exist. For non-equatorial climates the RAWREW and TAWTEW values are typically implemented with a transient seasonality to capture the annual lifecycle of vegetation or arable cropping successions. Alternatively, longer term crop rotations land-use change can be represented. In equatorial settings the variables are typically constant or altered to correspond to rain seasons.

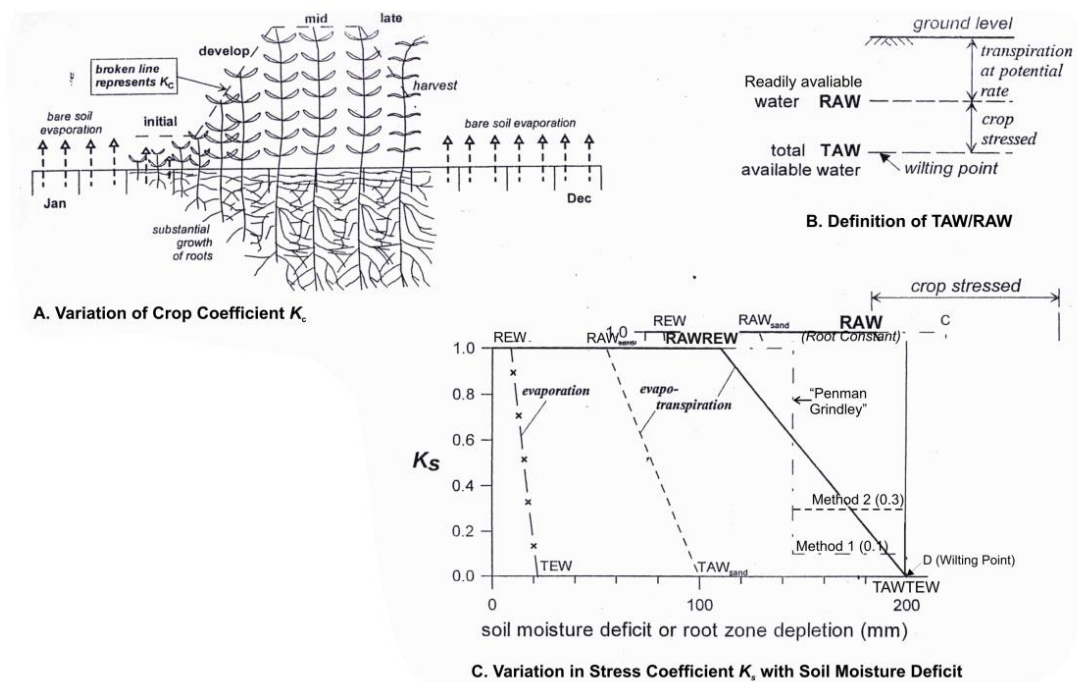


Figure 6.4 FAO 56 input parameter description. From WSP (2022), adapted from Allen et al. (1998).

The calculation of the soil moisture mechanism proceeds as follows on a per cell basis:

1. The volume of water entering the soil moisture mechanism (percolation into the root zone) is calculated as the remaining water from all preceding enabled processes
2. Calculate the K_s factor using the antecedent soil moisture deficit, and user defined RAWREW and TAWTEW parameters
3. The AE term is calculated from the product of K_c and PE which is subsequently multiplied by the K_s factor
 - a. If $k_c * PE > RAWREW$, then $AE = k_c * PE$
 - b. If $TAWTEW < k_c * PE \leq RAWREW$, then $AE = (k_c * PE) * k_s$
 - c. If $k_c * PE \leq TAWTEW$, then $AE = 0$
4. The soil moisture deficit for the current time period is calculated as the cumulative decrease caused by AE, and cumulative increase caused by percolation into the root zone

6.8 Percolation rejection

Percolation rejection is an optional mechanism added on the end of soil moisture accounting. It should be used to impose a limit to the amount of water that can infiltrate out of the base of the soil moisture store.

The input parameter is a percolation limit term applied either as a constant value or as a daily time-series. Percolation rejection may be thought of as a non-mass conservative upper limit to the amount of water allowed to exit the base of the soil moisture mechanism.

In most use-cases the value of percolation rejection can be set to an arbitrarily high value to effectively switch this process off and ensure that all water exiting the root zone percolates downwards towards the interflow mechanism.

6.9 Leakage below the root zone

The Leakage process is an optional mechanism designed to account for water leakage from infrastructure. Traditionally, this would be used to represent leakage from water mains and sewers within urban areas, as well as canals and any other leakage from infrastructure occurring below the root zone. As such, the water originating from leakage bypasses the soil moisture accounting and attenuation mechanisms described earlier and enters and is entered directly into the unsaturated zone. Finally, this process accepts a timeseries input of leakage, as well as a parameter determining the proportion of that leakage that can contribute to groundwater recharge.

6.10 Interflow and combined near surface lateral flow processes

Interflow is a weakly defined process that amalgamates the lateral flow mechanisms between the base of the rootzone and the saturated zone. Specifically, within SWAc interflow is the water which has percolated beyond the root zone that does not reach the saturated zone and migrates laterally to surface water courses. The process in SWAc typically results in attenuated rainfall-percolation events in surface water flow of the magnitude of days to a small number of weeks.

The simulated process employed to represent interflow is a 1st order decay function applied to a store. The decay proportion describes the proportion of water within the store that returns to surface water flow as interflow. The store also includes a flux limited pathway for water exiting the store as downwards infiltration recharge. Infiltration recharge egresses the interflow store and enters the recharge attenuation process or is added directly to recharge to groundwater. Both the decay proportion and the infiltration limit can be entered as a constant value or as a daily time-series.

The method implemented currently is consistent with the application in RRRR (Heathcote, et al., 2004).

An interflow bypass mechanism has also been included for which a further variable is required, namely a bypass fraction. The bypass volume is the product of the bypass fraction and water input to this mechanism. To exclude bypass the proportion can be set to zero.

A fourth variable, the initial store volume, describes the initial volume of water within the interflow store.

To summarise, the four input variables are:

- A decay function that describes interflow returning to surface water flow (interflow_decay)
- An infiltration limit describing how much water can infiltrate downwards out of the store towards recharge (infiltration_limit)
- A bypass proportion that describes how much input water is not subject to either decay or an infiltration limit (interflow_store_bypass)

- An initial store volume (init_interflow_store)

Future enhancement to the mechanism for infiltration recharge egress from the interflow store is scheduled. This development utilises a store volume dependent unsaturated zone function and will be an alternative to the commonly applied infiltration flux limit approach.

The calculation of the interflow mechanism proceeds as follows on a per cell basis:

1. *The bypass volume is calculated as the bypass fraction multiplied by the water input*
2. *The remaining water is directed through the interflow mechanism*
 - a. *If input volume < interflow infiltration limit, then infiltration = input volume*
 - b. *If input volume > interflow infiltration limit, then infiltration = interflow limit*
3. *Once the infiltration limit has been removed the product of the remaining water and the interflow decay proportion is directed back to surface water as interflow*
4. *Any remaining water is kept within the cumulative interflow store and applied to the next day's calculation*

6.11 Attenuation of recharge as it passes through the unsaturated zone

Attenuation of recharge prior to output of SWAc / input to MODFLOW is a process typically applied in areas of significant unsaturated zone thickness, low unsaturated conductivities and/or high pore storages. Recharge attenuation is currently represented as a 1st order decay function, which permits a proportion of the store volume to egress a recharge store each day. The released water is limited by a user specified constant maximum value.

6.12 Solute transport

There is generally a time lag between the release of contaminants at the ground surface (e.g. from fertilisers) and their arrival at the water table. Solute transport through the unsaturated zone is often a very slow process, and as such is difficult to predict when peak concentrations may occur due to historical leaching of contaminants. This time lag typically can constitute time periods spanning multiple years or even decades and, in the case of nitrate, is commonly referred to as the Nitrate Time Bomb (Wang & Burke, 2017) (Yang, et al., 2022).

Within SWAc, the solute mass leached from the ground surface on any day is apportioned to different flow components (e.g. percolation into the soil, surface runoff, etc.), based on calculated flow rates in SWAc as a proportion of HER (Hydrologically Effective Rainfall), i.e. the difference between precipitation and actual evapotranspiration. The apportionment of solute mass in the soil to the unsaturated zone is then based on the following assumptions: (i) the solute is mass conservative, (ii) the total amount leached on any day is based on HER for that that cell, (iii) the total mass leached is apportioned to near-surface and subsurface fluxes as a proportion of HER, and (iv) solute transport through the unsaturated zone is subject to advection and dispersion.

Advective travel time for mass-conservative solutes can be derived using estimates of mean velocity and thickness of the unsaturated zone (Wang & Burke, 2017) (Yang, et al., 2022). However, accounting only for advection would indicate the sudden arrival of solute at the water table within a single day, potentially several years after it was leached from the shallow subsurface during a heavy rainfall event. This “spike” of solute mass is considered unrealistic and conceptually, would imply an entirely homogenous unsaturated zone, with no variations in hydraulic conductivity along its travel path.

As such, a set of bell curves can be estimated. Each curve represents a breakthrough of solute at the water table from one leaching event. Those contributions of leached mass are then combined, as

shown in Figure 6.5 (using nitrate as an example), to give a total mass reaching the water on any given day. That total mass (in g) is then combined with the modelled recharge volume for that day (in m³) to derive a recharge nitrate concentration (in g/m³, equivalent to mg/l).

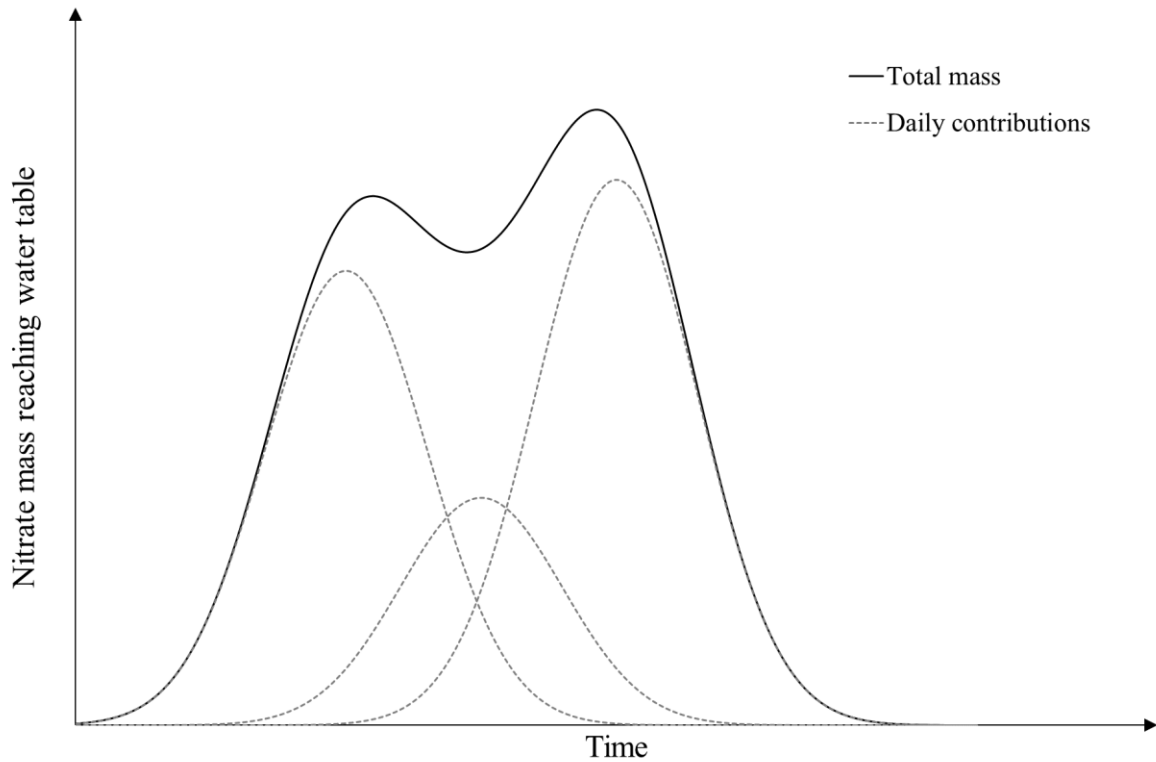


Figure 6.5. Example schematic showing cumulative nitrate mass reaching the water table from multiple events

For each leaching event, the Cumulative Distribution Function (CDF) for a population with log-normal distribution is given by Equation (6.1):

$$F(K) = \frac{1}{2} \left[1 + \operatorname{erf} \left(-\frac{\log_{\alpha}(K) - \mu}{\sigma\sqrt{2}} \right) \right] \quad (6.1)$$

Where K is the hydraulic conductivity (m/day), erf is the error function, α is the base of the logarithm, μ is the mean of the variable's logarithm, and σ is the standard deviation of the variable's logarithm.

To characterise solute transport through the unsaturated zone, it is necessary to know the proportion of water from a given recharge event that arrives on each subsequent day. As such, Equation (6.1) must be reformulated in terms of travel time. If the depth to groundwater, DTW [m], is known (e.g. based on modelled long-term average groundwater levels), travel time, t [d], through the unsaturated zone can be derived from groundwater velocity, v [m/d], using Equation (6.2):

$$v = \frac{DTW}{t} \quad (6.2)$$

The groundwater velocity is given by Equation (6.3):

$$v = -\frac{K}{n_e} \frac{dh}{dl} \quad (6.3)$$

Where n_e is the effective porosity and $\frac{dh}{dl}$ is the hydraulic gradient.

Assuming a vertical hydraulic gradient and ignoring the sign convention for flow direction, Equation (6.3) reduces to Equation (6.4):

$$v = -\frac{K}{n_e} \quad (6.4)$$

Due to the effects of air-filled voids in the pore space, hydraulic conductivity in the unsaturated zone K_{USZ} may be several orders of magnitude lower than K . Mean values of these two parameters can be compared using Equation (6.5):

$$\bar{K} = a \overline{K_{USZ}} \quad (6.5)$$

Where the coefficient $a \gg 1$.

Combining Equations (6.1), (6.2), (6.4), and (6.5) allows the CDF to be expressed in terms of inverse travel time $\frac{1}{t}$ as show in Equation (6.6):

$$F\left(\frac{1}{t}\right) = \frac{1}{2} \left[1 + \operatorname{erf} \left(-\frac{\log_a \left(a n_e DTW \frac{1}{t} \right) - \mu}{\sigma \sqrt{2}} \right) \right] \quad (6.6)$$

Finally, the proportion of applied solute $Sprop_i$ reaching the water table i days after leaching from the soil is calculated using Equation (6.7):

$$Sprop_i = F\left(\frac{1}{t_i}\right) - F\left(\frac{1}{t_{i-1}}\right) \quad (6.7)$$

6.13 Surface water attenuation

Surface flow is an output from each cell and is the sole lateral link within the SWAc model. The surface water model topological routing network links each cell to its downstream neighbour. This provides an input of water to downstream cells which can be subject to local runoff recharge processes. In addition, for larger catchments the time for water to pass down through the system can exceed the time stepping of the model. To account for the downstream time of concentration and the smoothing of upstream events as they migrate to the base of the lower reaches of the catchment, a surface flow attenuation process is included.

6.14 Surface water ponding

This incorporates a surface water attenuation store, which is filled by: upstream surface water flow (total discharge, including runoff in upstream cells, minus abstractions); precipitation falling directly on the surface water body; runoff (both local rapid runoff and rejected recharge); and interflow. If the store exceeds the user-specified maximum ponding depth, the entirety of that excess is passed downstream as overspill. After accounting for open-water evaporation, remaining water in the store that exceeds a user-specified activation threshold is apportioned in the following order: passed downstream, based on a user-specified release percentage; infiltrated to groundwater, based on a user-specified infiltration percentage. Finally, infiltrating groundwater can be applied either as direct recharge, or attenuated in the unsaturated zone, based on a user-specified direct recharge percentage.

The user should also specify the proportion of each cell subject to ponding, as this controls the impact of inputs on ponding depth, and the respective impacts of downstream release and infiltration on stream flow and recharge. Essentially, a small surface water body within a large catchment will fill up rapidly when runoff occurs.

With the exception of maximum ponding depth, area subject to ponding, and initial ponding depth, which must be specified as time invariant quantities, all other inputs can be varied on a monthly basis,

to reflect seasonal variations in evaporation, flow rate and connectivity to the underlying groundwater system.

6.15 Surface water routing

The movement of runoff and interflow to watercourses is dictated by the local topography, which also dictates the direction of flow within the river network. SWAc requires information on the topology of the model mesh, so that each cell is connected to its upstream and downstream neighbours. In addition, information on watercourse morphology (e.g. stream bed width) and hydraulic properties (i.e. stream bed conductivity) is used to generate suitably formatted SFR files for input to MODFLOW.

6.16 Surface water discharge

Anthropogenic discharges to surface water are added to the total stream flow in SWAc outputs. Inputs comprise a list of nodes where discharges occur and a matrix with one time-series column for each of these locations. The user has the option to specify time series inputs as daily, weekly or monthly average values in m^3/d . In common with standard conventions for modelling, time series discharges should be entered as positive values.

6.17 Surface water abstraction

Abstractions from surface water are subtracted from total stream flow in SWAc outputs. Similar to anthropogenic discharges, inputs comprise a list of nodes where abstractions occur and a matrix with one time-series column for each of these locations. The user has the option to specify time series inputs as daily, weekly or monthly average values in m^3/d . In common with standard conventions for modelling, time series abstractions should be entered as negative values.

7 Model master control file and input methods

7.1 Input file structure

To run SWAc a single input control file is required. This file contains references to other files, parameters and information that describes the model that is to be run. A template input control file is presented in Appendix A.

The format of the file is based on the YAML language scheme, which is human readable and readily parsed by Python codes. Further information on YAML can be found here: <http://docs.ansible.com/ansible/YAMLSyntax.html>

Lines prefixed with a '#' are not read by the program. Similarly, blank lines are ignored. Therefore, comments can be added into the file.

The program reads the master control file and assures all required input is present. If mandatory items are missing the program will exit and comment on required files output to both the screen and a log file.

In reading the master control file the program looks for keywords, for example "run_name:" then reads the information to the right of this as a keyword pair. The keyword pair can be in the format of text or a file path. If the user keeps all input files in the same directory as the master control file, then file paths need only consist of the file name itself, including the file extension.

File paths will be checked, file formats tested, and variable values checked to assure the correct format and possible range of values for that process. As the program is keyword based, the reading of the control file is much more 'forgiving' and the keyword pairs can actually be arranged in any order, as long as they are properly matched.

The layout of the template master control file provided with SWAc is shown below. The rest of the chapter discusses all sections of the control file in order and describes in detail all the user inputs required; this information is provided in summary format in Appendix B. The main sections of the control file are as follows:

- Global inputs
- Climatic Time-series datasets
- Main model processes
- Anthropogenic processes
- Attenuation
- Output parameters

7.2 Global inputs

These user inputs include the input file version, model name, timing, model mesh geometry, and computer processing resources to be assigned to the model.

The keyword version describes the version of the input file. The pair to this keyword has to be provided as text representing an integer. The latest version of the input file is version 2. If omitted, version 1 is assumed.

The model name is assigned with the required keyword `run_name`, the user provides the pair to this keyword as text. All model output files will start with the name provided here. Certain characters are not allowed in the model name, such as colons and quotation marks. Any commas, or full stops in the file name will be returned as an underscore in model output file names.

The required keyword `num_nodes` describes the number of nodes in the model mesh. The pair to this keyword has to be provided as text representing an integer.

The required keyword `node_areas` provides a list of areas for each model node. The pair to this keyword is a link to either a CSV file or a YAML file. In the case of a csv file the format is shown in Figure 7.1. Column A is the node number integer. Column B is the area of the node in m² represented either by an integer or a float (decimal number). The example provided is a model consisting of ten nodes of variable area size.

	A	B
1	1	74.8
2	2	7.67
3	3	82.17
4	4	160.44
5	5	197.22
6	6	92.2
7	7	221.54
8	8	163.41
9	9	127.99
10	10	296.5885

Figure 7.1 Node areas CSV input example

The corresponding YAML file option is shown in Figure 7.2. The YAML file can contain multiple comments starting with the `#` character that will not be read by SWAc but may provide the user with useful metadata. Excluding comments, the first machine readable line of the YAML file contains the keyword `node_areas` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the node area as a float. The indentation and space obligations are to preserve the Python dictionary data structure, often called an associative array. Note the order of nodes does not need to be sequential as long as all nodes are included and the total number of nodes matches the `num_nodes` keyword pair specification.

```
# Area of each node.
node_areas:
  7: 221.54
  8: 163.41
  9: 127.99
  1: 74.8
  2: 7.67
  3: 82.17
  4: 160.44
  5: 197.22
  10: 296.5885
  6: 92.2
```

Figure 7.2 Node areas YAML input example

The optional keyword `node_xy` lists the special coordinates of every model node centroid. This information however is not used by the current version of SWAc for its calculations.

Note that in SWAc the model mesh can be of any geometry, and node numbers do not have to follow any particular spatial order.

The start date of the model is assigned with the required keyword `start_date`. The pair to this keyword has to be provided as text in the format YYYY-MM-DD.

The length of each time period is assigned via the required keyword `time_periods`. The pair to this keyword is a link to either a CSV file or a YAML file. In the case of a CSV, the time period file should contain two columns with no headers as shown in Figure 7.3, and each row represents a time period. Column A represents the start of each time period in days from the start date, and Column B represents the end of each period in days from the start date. In the example shown in Figure 7.3, the model has three periods. Period 1 consists of 31 days (days 1-31), period 2 contains 28 days (days 31-59), and period 3 contains 31 days (days 59-90). This is the typical setup for running SWAc on monthly time periods.

	A	B
1	1	31
2	31	59
3	59	90

Figure 7.3 Time period CSV input example

The corresponding YAML file option is shown in Figure 7.4. The YAML file can contain multiple comments starting with the `#` character that will not be read by SWAc but may provide the user with useful metadata. Excluding comments, the first machine readable line of the YAML file contains the keyword `time_periods` followed by a colon. The following lines are indented by four spaces for readability, and then contain an obligatory hyphen followed by a single space followed by a Python list in square brackets. Item 1 of the list is the start of each time period in days, item 2 of the list is the end of each period in days.

```
# Output data will be aggregated according to these time windows.
# Columns are
# 1) start
# 2) end
time_periods:
  - [1, 31]
  - [31, 59]
  - [59, 90]
```

Figure 7.4 Time period YAML input example

The optional keyword `num_cores` controls the number of processing threads used to run the SWAc model. The keyword pair is an integer up to the number of threads available on the computing platform being used to run the model. A minus value can also be entered, for example -1 will use all available threads except one. Assigning the number of threads to be used can improve runtime efficiency, however using too many can actually decrease runtime performance in some cases. A trial-and-error approach is recommended when using this option. If left blank, all computer threads will be used.

The keyword `output_recharge` controls whether a MODFLOW compatible RCH file will be output at the end of the model run. Accepted values are “true” or “false”. If left blank, it will be set to “true”.

The keyword `output_individual` can be used to output a water balance file for specific model nodes. Accepted values are “none”, or comma separated values or value ranges as strings, i.e. they need to

be enclosed within quotation marks (e.g. "1, 3, 68-83, 5-7, 6"). Overlapping ranges are also accepted. If not specified, it will be set to "none".

The keyword `irchcb` specifies the IRCHCB parameter in MODFLOW, i.e. the unit number in MODFLOW RCH file for output to cell-by-cell file. Accepted values are integers.

The keyword `nodes_per_line` specifies the number of nodes per line of the MODFLOW RCH file. Accepted values are integers.

The keyword `recharge_node_mapping` can be used to map the SWAc nodes to specific MODFLOW nodes for the generation of the MODFLOW RCH file. If not specified, it is assumed that recharge will be assigned the upper most active layer of MODFLOW. The keyword pair can be a link to either a CSV or a YAML file. In the case of CSV, the format is shown in Figure 7.5. Column A lists the SWAc node number, and Column B list the corresponding MODFLOW node number. Both columns A and B should be integers.

	A	B
1	1	1
2	2	2
3	3	3
4	4	0
5	5	5
6	6	6
7	7	7
8	8	8
9	9	99
10	10	10

Figure 7.5. Recharge node mapping CSV example

In the case of a YAML file the format is shown in **Error! Not a valid bookmark self-reference..** The first line of the YAML file contains the keyword `recharge_node_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the SWAc node number with a colon, another obligatory space, and a list with the MODFLOW node number as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
recharge_node_mapping:
  1: [1]
  2: [2]
  3: [3]
  4: [0]
  5: [5]
  6: [6]
  7: [7]
  8: [8]
  9: [99]
  10: [10]
```

Figure 7.6. Recharge node mapping YAML example

The keyword `output_fac` defined the factor that the outputs will be multiplied by and can be used for output unit conversion. Accepted values are floats. If not specified, it will be set to 1.0.

The keyword `spatial_output_date` can be used to specify balance file outputs for all SWAc nodes for a specific day. Accepted values are text in the format YYYY-MM-DD. It can also accept “none” for no output, or “mean” for outputting the average balance across the simulation period. If not specified, it’ll be set to “none”.

7.3 Spatially defined static properties

The model is a vertical workflow per cell. Each workflow is repeated in each and every model cell, however the parameterisation of the component processes is likely to differ across space. All model inputs use a consistent methodology for applying properties across space. Either Zones can be utilised or explicitly defined values for each cell.

For the zone method, 3 files are required: (i) a zone mapping file specifying the zone number each cell is assigned to, (ii) a zone name file, specifying an arbitrary name for that zone, and (iii) a zone parameter file which provides the input variable to control the simulated process. In all parts of SWAc the zone mapping and name files use the same format and syntax. The parameter files differ by process.

Zone spatial mapping files can be input in CSV or YAML formats, and example of which is presented in Figure 7.7 (left and right, respectively).

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	3
7	7	3
8	8	3
9	9	4
10	10	4
11		

1	rapid_runoff_zone_mapping:
2	1: 1
3	2: 1
4	3: 1
5	4: 2
6	5: 2
7	10: 4
8	6: 3
9	7: 3
10	8: 3
11	9: 4
12	

Figure 7.7 Examples of spatial zone mapping files in CSV (left) and YAML (right) format

Zone name files can also be input in CSV or YAML formats, and example of which is presented in Figure 7.8 (left and right, respectively).

	A
1	Catchment1
2	Catchment2
3	Catchment3
4	Catchment4
5	

1	rapid_runoff_zone_names:
2	1: Catchment1
3	2: Catchment2
4	3: Catchment3
5	4: Catchment4
6	

Figure 7.8 Examples of spatial zone name files in CSV (left) and YAML (right) format

The format of zone parameter file differ for each process. An example of the rapid runoff parameter file is shown in Figure 7.9. This is the first 2 of 4 runoff zones parameters.

```

0 10 20 30 40 50 60 70 80 90
1 rapid_runoff_params: [
2
3     {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
4       class_ri: [5, 10, 20, 25, 30, 35, 999],  # Rainfall intensity [mm] classes
5       values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
6                [0.18, 0.14, 0.08, 0.02, 0.05],
7                [0.19, 0.14, 0.08, 0.03, 0.1],
8                [0.2, 0.15, 0.09, 0.04, 0.1],
9                [0.22, 0.16, 0.1, 0.05, 0.2],
10               [0.24, 0.17, 0.11, 0.06, 0.2],
11               [0.26, 0.18, 0.12, 0.08, 0.2]]},
12
13     {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
14       class_ri: [5, 10, 20, 25, 30, 35, 999],  # Rainfall intensity [mm] classes
15       values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
16                [0.18, 0.14, 0.08, 0.02, 0.05],
17                [0.19, 0.14, 0.08, 0.03, 0.1],
18                [0.2, 0.15, 0.09, 0.04, 0.1],
19                [0.22, 0.16, 0.1, 0.05, 0.2],
20                [0.24, 0.17, 0.11, 0.06, 0.2],
21                [0.26, 0.18, 0.12, 0.08, 0.2]]},

```

Figure 7.9 Example input file for rapid runoff parameters

To explicitly define a property to each cell the file format shown in Figure 7.10 can be used. In this example we input all properties controlling the interflow process. The properties are defined in the same file as the spatial mapping. There are no zones and no need for a zone name file.

	A	B	C	D	E
1	1	10	0.05	0.7	0.05
2	2	10	0.05	0.7	0.05
3	3	10	0.05	0.7	0.05
4	4	10	0.05	0.7	0.05
5	5	10	0.05	0.7	0.05
6	6	10	0.05	0.7	0.05
7	7	10	0.05	0.7	0.05
8	8	10	0.05	0.7	0.05
9	9	10	0.05	0.7	0.05
10	10	10	0.05	0.7	0.05

```

0 10 20 30
1 # Columns are:
2 # 1) "init_interflow_store"
3 # 2) "store_bypass"
4 # 3) "infiltration"
5 # 4) "interflow_to_rivers"
6 interflow_params:
7   1: [10.0, 0.05, 0.7, 0.05]
8   2: [10.0, 0.05, 0.7, 0.05]
9   4: [10.0, 0.05, 0.7, 0.05]
10  5: [10.0, 0.05, 0.7, 0.05]
11  6: [10.0, 0.05, 0.7, 0.05]
12  7: [10.0, 0.05, 0.7, 0.05]
13  8: [10.0, 0.05, 0.7, 0.05]
14  9: [10.0, 0.05, 0.7, 0.05]
15 10: [10.0, 0.05, 0.7, 0.05]
16  3: [10.0, 0.05, 0.7, 0.05]

```

Figure 7.10. Interflow input parameters CSV (left) and YAML (right) example

7.4 Climatic time series

Climatic timeseries input data include rainfall, potential evapotranspiration, temperature, and windspeed. Rainfall is mandatory, Potential Evapotranspiration is required for Canopy, Sublimation or Soil Moisture accounting processes. Temperature is required for snow processes, solar flux and windspeed are required in the snow pack-sublimation process.

All time variant input files are assumed to be daily temporal resolution and all timeseries input files are assigned to the spatial zones.

The time series input file itself is a simple comma separated values (CSV) file, with rows being days 1 to the total days in the simulation and columns being zones 1 to the total number of zones. Input validation is undertaken to assure there are the correct number of days and zones in the file and the values are of the appropriate type.

The timeseries data are required to be mapped onto the model grid using a separate mapping input file. The mapping file facilitates 3 methods for spatially distributing time variant data:

1. Zones – where all cells within a zone are given the timeseries assigned to that zone

2. Perturbation factors – All cells are classed as zone 1 and assigned the sole timeseries value. However, each cell is assigned a static, steady state perturbation factor. The rainfall timeseries is multiplied by this factor.
3. Combined Zones and Perturbation factors – multiple zones applied and the assigned timeseries for each cell is additionally factored.

The latter 2 methods can be useful in representing spatial change in patterns. For example, radar data to spatially distribute rain gauge data, rainfall, or topographic data, or aspect data for developing relationships between topography and the model input.

There are 2 input formats, CSV (comma separated values) and YAML, and either one can be used. SWAc will automatically detect which input type is being used during the input validation

CSV input files for timeseries mapping require 3 columns: Column A lists the node number, Column B lists the zone number, and Column C lists the perturbation factor. An example of CSV input for timeseries mapping is shown in Figure 7.12.

	A	B	C
1	1	1	1
2	2	1	1
3	3	1	1
4	4	1	1
5	5	2	1
6	6	2	1
7	7	2	1
8	8	2	1
9	9	2	1
10	10	2	1
11			

Figure 7.11 Timeseries input mapping file CSV example

YAML input files follow the format shown in Figure 7.12. The YAML file can contain multiple comments starting with the # character that will not be read by SWAc, but may provide the user with useful metadata. Excluding comments, the first machine readable line of the YAML file contains a keyword relating to the timeseries being imported, followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, the zone number as an integer, and the perturbation factor as a float. The indentation and space obligations are to preserve the Python dictionary data structure, often called an associative array. Note the order of nodes does not need to be sequential, as long as all nodes are included, and the total number of nodes matches the num_nodes keyword pair specification.



```

1 # Columns are
2 # 1) zone
3 # 2) proportion
4 pe_zone_mapping:
5   1: [1, 1.0]
6   2: [1, 1.0]
7   3: [1, 1.0]
8   4: [1, 1.0]
9   5: [2, 1.0]
10  10: [2, 1.0]
11  6: [2, 1.0]
12  7: [2, 1.0]
13  8: [2, 1.0]
14  9: [2, 1.0]
15
16

```

Figure 7.12 Timeseries input mapping file YAML example

7.5 Main model processes

For each model process described in this Section, the input files required are listed and the file formats are specified. Each process is switched on with the keyword “*enabled*” written directly into the master control file, unwanted processes are switched off with the keyword “*disabled*”. Care should be taken as certain processes act as necessary precursors to others, for example runoff recharge cannot occur if the rapid runoff process is disabled. All processes need to be listed in the master control file and specified as either “*enabled*” or “*disabled*”.

7.5.1 Canopy interception

Four input files are required for this process:

- A spatial mapping file (canopy_zone_mapping)
- A zone name file (canopy_zone_names)
- Parameterise a throughfall proportion for each zone (free_throughfall)
- Parameterise a maximum storage value in mm for each zone (max_canopy_storage)

The spatial mapping file keyword is canopy_zone_mapping, the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.13. Column A lists the node numbers, and Column B lists the canopy mapping zone to which the node is assigned. Both column A and column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	10	2
7	6	2
8	7	3
9	8	2
10	9	2

Figure 7.13 Canopy Zone Spatial Mapping CSV example

In the case of a YAML file, the format is shown in Figure 7.14. The first line of the YAML file contains the keyword `canopy_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the canopy mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
canopy_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 2
  6: 2
  7: 3
  8: 2
  9: 2
```

Figure 7.14 Canopy Zone Spatial Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The canopy zone name file keyword is `canopy_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.15. The first line of the YAML file contains the keyword `canopy_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
canopy_zone_names:
  1: Catchment1
  2: Catchment2
  3: Catchment3
```

Figure 7.15 Canopy Zone Name File YAML example

The `free_throughfall` keyword pair is a link to a CSV or YAML file containing a list of zone numbers as integers and a corresponding throughfall proportion as floats. In the case of a CSV file, the format is shown in Figure 7.16. Column A lists the zone numbers, and Column B lists the throughfall proportion.

	A	B
1	1	0.99
2	2	0.99
3	3	0.99

Figure 7.16 Canopy free_throughfall CSV example

In the case of a YAML file, the format is shown in Figure 7.14. The first line of the YAML file contains the keyword `free_throughfall` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the throughfall fraction. The indentation and space obligations are to preserve the Python dictionary data structure.

```
free_throughfall:
  1: 0.99
  2: 0.99
  3: 0.99
```

Figure 7.17 Canopy free_throughfall YAML example

The max_canopy_storage keyword pair is a link to a CSV or YAML file containing a list of zone numbers as integers and corresponding storage limits as floats. In the case of a CSV file the format is shown in Figure 7.18. Column A lists the zone numbers, and Column B lists the storage limits.

	A	B
1	1	1.00
2	2	1.05
3	3	1.00

Figure 7.18 max_canopy_storage CSV example

In the case of a YAML file the format is shown in Figure 7.19. The first line of the YAML file contains the keyword max_canopy_storage followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the storage as a float. The indentation and space obligations are to preserve the Python dictionary data structure.

```
max_canopy_storage:
  1: 1.00
  2: 1.05
  3: 1.00
```

Figure 7.19 max_canopy_storage YAML example

7.5.2 Snow melt

Simple snow process

Three input files are required for this process:

- A spatial mapping file (temperature_zone_mapping)
- A zone name file (temperature_zone_names)
- Parameterise an initial snowpack, snowfall, and snowmelt thresholds for each zone (snow_params_simple)

The spatial mapping file keyword is temperature_zone_mapping, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.20. Column A lists the node numbers. Column B lists the temperature mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	1
5	5	2
6	6	2
7	7	2
8	8	2
9	9	2
10	10	2
11		

Figure 7.20 Temperature Zone Mapping CSV example

In the case of a YAML file the format is shown in Figure 7.21. The first line of the YAML file contains the keyword `temperature_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the temperature mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
temperature_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 1
  5: 2
  10: 2
  6: 2
  7: 2
  8: 2
  9: 2
```

Figure 7.21 Temperature Zone Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The temperature zone name file keyword is `temperature_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.22. The first line of the YAML file contains the keyword `temperature_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
temperature_zone_names:
  1: Catchment1
  2: Catchment2
```

Figure 7.22 Temperature Zone Names YAML example

The main parameter input file for the simple snow process is represented by the `snow_params_simple` keyword. The keyword pair is a link to a CSV or YAML file, containing a list of node numbers, the starting snowpack, the snowfall threshold, and a snowmelt threshold as integers. In the case of a CSV file, the format is shown in Figure 7.23. Column A lists the node numbers, Column B lists the starting snowpack, Column C lists the temperature threshold for snowfall, and Column D lists the temperature threshold for snowmelt.

	A	B	C	D
1	1	100	1	-2
2	2	100	1	-2
3	3	100	1	-2
4	4	100	1	-2
5	5	100	1	-2
6	6	100	1	-2
7	7	100	1	-2
8	8	100	1	-2
9	9	100	1	-2
10	10	100	1	-2
11				

Figure 7.23 `snow_params_simple` CSV example

In the case of a YAML file the format is shown in Figure 7.24. The first line of the YAML file contains the keyword `snow_params_simple` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, the starting snowpack, the snowfall threshold, and the snowmelt threshold as integers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
snow_params_simple:
  1: [100, 1, -2]
  2: [100, 1, -2]
  4: [100, 1, -2]
  5: [100, 1, -2]
  6: [100, 1, -2]
  7: [100, 1, -2]
  8: [100, 1, -2]
  9: [100, 1, -2]
  10: [100, 1, -2]
  3: [100, 1, -2]
```

Figure 7.24 `snow_params_simple` YAML example

Complex snow process

Three input files are required for this process:

- A spatial mapping file for maximum daily temperature (`tmax_c_zone_mapping`)
- A zone name file for maximum daily temperature (`tmax_c_zone_names`)
- A spatial mapping file for minimum daily temperature (`tmin_c_zone_mapping`)
- A zone name file for minimum daily temperature (`tmin_c_zone_names`)

- A spatial mapping file for average daily windspeed (windsp_zone_mapping)
- A zone name file for average daily windspeed (windsp_zone_names)
- Parameterise the snowmelt calculation parameters (snow_params_complex)

The maximum temperature spatial mapping file keyword is `tmax_c_zone_mapping`, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.25. Column A lists the node numbers. Column B lists the maximum temperature mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	1
5	5	1
6	6	1
7	7	1
8	8	1
9	9	1
10	10	1
11		

Figure 7.25. Maximum daily temperature zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.26. The first line of the YAML file contains the keyword `tmax_c_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the maximum temperature mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# Columns are
# 1) zone
tmax_c_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 1
  5: 1
  10: 1
  6: 1
  7: 1
  8: 1
  9: 1
```

Figure 7.26. Maximum daily temperature zone mapping YAML example

The maximum daily temperature zone name file keyword is `tmax_c_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.27. The first line of the YAML file contains the keyword `tmax_c_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# tmax_c zone names.
tmax_c_zone_names:
  1: Catchment1
```

Figure 7.27 Maximum daily temperature zone names YAML example

The minimum temperature spatial mapping file keyword is `tmin_c_zone_mapping`, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.28. Column A lists the node numbers. Column B lists the minimum temperature mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	1
5	5	1
6	6	1
7	7	1
8	8	1
9	9	1
10	10	1
11		

Figure 7.28. Minimum daily temperature zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.29. The first line of the YAML file contains the keyword `tmin_c_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the minimum temperature mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# Columns are
# 1) zone
tmin_c_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 1
  5: 1
  10: 1
  6: 1
  7: 1
  8: 1
  9: 1
```

Figure 7.29. Minimum daily temperature zone mapping YAML example

The minimum daily temperature zone name file keyword is `tmin_c_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.30. The first line of the YAML file contains the keyword `tmin_c_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# tmin_c zone names.
tmin_c_zone_names:
  1: Catchment1
```

Figure 7.30 Minimum daily temperature zone names YAML example

The average daily windspeed spatial mapping file keyword is `windsp_zone_mapping`, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.31. Column A lists the node numbers. Column B lists the average daily windspeed mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	1
5	5	1
6	6	1
7	7	1
8	8	1
9	9	1
10	10	1
11		

Figure 7.31. Average daily windspeed zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.29. The first line of the YAML file contains the keyword `windsp_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the average daily windspeed mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# Columns are
# 1) zone
windsp_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 1
  5: 1
  10: 1
  6: 1
  7: 1
  8: 1
  9: 1
```

Figure 7.32. Average daily windspeed zone mapping YAML example

The average daily windspeed zone name file keyword is `windsp_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.33. The first line of the YAML file contains the keyword `windsp_c_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# windsp zone names.
windsp_zone_names:
  1: Catchment1
```

Figure 7.33 Average daily windspeed zone names YAML example

The main parameter input file for the complex snow process is represented by the snow_params_complex keyword. The keyword pair is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.34. Column A lists the node numbers as integers, Column B lists the latitude in degrees as floats, Column C lists the slope as vertical height over horizontal distance as floats, Column D lists the aspect in degrees as floats, Column E lists the height of temperature measurement above ground in metres as floats, Column F lists the height of wind speed measurement above ground in metres as floats, Column G lists the ground albedo as floats, Column H lists the surface emissivity ratio as floats, Column I lists the proportion of tree cover as floats, Column J lists the starting snow depth in metres as floats, and Column K lists the starting snow density in kilograms per cubic metre as floats.

	A	B	C	D	E	F	G	H	I	J	K
1	1	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
2	2	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
3	4	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
4	5	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
5	6	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
6	7	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
7	8	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
8	9	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
9	10	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450
10	3	7.55	0.05	4.79	20	100	0.25	0.95	1	0	450

Figure 7.34. snow_params_complex CSV example

In the case of a YAML file the format is shown in Figure 7.35. The first line of the YAML file contains the keyword snow_params_complex followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, the latitude in degrees as floats, the slope as vertical height over horizontal distance as floats, the aspect in degrees as floats, the height of temperature measurement above ground in metres as floats, the height of wind speed measurement above ground in metres as floats, the ground albedo as floats, the surface emissivity ratio as floats, the proportion of tree cover as floats, the starting snow depth in metres as floats, and the starting snow density in kilograms per cubic metre as floats.

```
snow_params_complex:
  1: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  2: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  4: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  5: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  6: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  7: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  8: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  9: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  10: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
  3: [7.55, 0.05, 4.79, 20.0, 100.0, 0.25, 0.95, 1.0, 0.0, 450.0]
```

Figure 7.35 snow_params_complex YAML example

7.5.3 Rapid runoff

Three input files are required for this process:

- A spatial mapping file (rapid_runoff_zone_mapping)
- A zone name file (rapid_runoff_zone_names)
- Parameterise runoff coefficient proportion for each zone (rapid_runoff_params)

The spatial mapping file keyword is rapid_runoff_zone_mapping, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.36. Column A lists the node numbers. Column B lists the rapid runoff mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	3
7	7	3
8	8	3
9	9	4
10	10	4

Figure 7.36 Rapid Runoff Zone Mapping CSV example

In the case of a YAML file the format is shown in Figure 7.37. The first line of the YAML file contains the keyword rapid_runoff_zone_mapping followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the rapid runoff mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
rapid_runoff_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 4
  6: 3
  7: 3
  8: 3
  9: 4
```

Figure 7.37 Rapid Runoff Zone Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the num_nodes keyword pair specification.

The rapid runoff zone name file keyword is rapid_runoff_zone_names, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.38. The first line of the YAML file contains the keyword rapid_runoff_zone_names followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary

zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
rapid_runoff_zone_names:
  1: Catchment1
  2: Catchment2
  3: Catchment3
  4: Catchment4
```

Figure 7.38 Rapid Runoff Zone Name File YAML example

The main parameter input file for the rapid runoff mechanism is represented by the `rapid_runoff_params` keyword. The keyword pair is a link to a YAML file, the format of which is shown in Figure 7.39. In this example, there are four rapid runoff zones, each represented by a Python dictionary, as indicated by the four sets of curly brackets. Each of the four dictionaries are packaged within a high-level Python list called `rapid_runoff_params`, as indicated by the keyword on first line of the file followed by a colon and an open square bracket, that becomes closed on the last line of the file. The file structure can be broken down as follows:

- A single high-level list called `rapid_runoff_params` that consists of 4 dictionaries, each dictionary representing a rapid runoff zone, and indicated within a set of curly brackets.
- Each dictionary contains 3 key-value pairs, the keys are as follows: `class_smd` (soil moisture deficit classes), `class_ri` (rainfall intensity classes), and `values` (the runoff coefficients).
- The `class_smd` pair is a list of integers or floats, as indicated with square brackets. This list represents the number of user-defined soil moisture deficit classes (in mm). The `class_ri` is also a list of integer or float values representing the user-defined rainfall intensity classes to be used (in mm). The comments to the right of each of these lists (indicated by the # character) are included for user clarity only, their omission will not impact on the ability of the model to run. Note how the last value of both lists is 999 to indicate the highest class level. The list values represent the upper limit of each class. For example, the soil moisture deficit class of 5 represents all soil moisture values from 0 to 5 mm, similarly the rainfall intensity value 999 represents all daily rainfall totals ranging from 20.01 mm to 999 mm.
- The `values` pair is a list containing a further 7 lists (referred to herein as sub-lists). Each sub-list contains 5 proportions as floats (the proportions are the runoff coefficients). A value of 0.16 represents a 16% runoff coefficient and so on. 7 sub-lists are present because we have 7 rainfall intensity classes. Each sub-list contains 5 runoff coefficients because we have 5 SMD classes. The user can change the number of RI and SMD classes used but the number of sub-lists and number of items within each sub-list will also need match the new number of classes assigned.
- As an example, the third runoff coefficient within rapid runoff zone 1, value sub-list 1, is represented by the float 0.06. This value represents a 6% runoff coefficient assigned to the first RI class of 0-5mm/day and the third SMD class 10.01 – 15mm.
- Similarly, the first runoff coefficient within rapid runoff zone 2, value sub-list 2, is the float 0.18. This value represents an 18% runoff coefficient assigned to the second RI class of 5.01-10mm/day and the first SMD class 0 – 5mm.

```

rapid_runoff_params: [
  {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
    class_ri: [5, 10, 20, 25, 30, 35, 999], # Rainfall intensity [mm] classes
    values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
              [0.18, 0.14, 0.08, 0.02, 0.05],
              [0.19, 0.14, 0.08, 0.03, 0.1],
              [0.2, 0.15, 0.09, 0.04, 0.1],
              [0.22, 0.16, 0.1, 0.05, 0.2],
              [0.24, 0.17, 0.11, 0.06, 0.2],
              [0.26, 0.18, 0.12, 0.08, 0.2]]},

  {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
    class_ri: [5, 10, 20, 25, 30, 35, 999], # Rainfall intensity [mm] classes
    values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
              [0.18, 0.14, 0.08, 0.02, 0.05],
              [0.19, 0.14, 0.08, 0.03, 0.1],
              [0.2, 0.15, 0.09, 0.04, 0.1],
              [0.22, 0.16, 0.1, 0.05, 0.2],
              [0.24, 0.17, 0.11, 0.06, 0.2],
              [0.26, 0.18, 0.12, 0.08, 0.2]]},

  {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
    class_ri: [5, 10, 20, 25, 30, 35, 999], # Rainfall intensity [mm] classes
    values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
              [0.18, 0.14, 0.08, 0.02, 0.05],
              [0.19, 0.14, 0.08, 0.03, 0.1],
              [0.2, 0.15, 0.09, 0.04, 0.1],
              [0.22, 0.16, 0.1, 0.05, 0.2],
              [0.24, 0.17, 0.11, 0.06, 0.2],
              [0.26, 0.18, 0.12, 0.08, 0.2]]},

  {class_smd: [5, 10, 15, 20, 999],          # Antecedent Soil Moisture Deficit classes
    class_ri: [5, 10, 20, 25, 30, 35, 999], # Rainfall intensity [mm] classes
    values: [[0.16, 0.12, 0.06, 0.02, 0.02], # Values
              [0.18, 0.14, 0.08, 0.02, 0.05],
              [0.19, 0.14, 0.08, 0.03, 0.1],
              [0.2, 0.15, 0.09, 0.04, 0.1],
              [0.22, 0.16, 0.1, 0.05, 0.2],
              [0.24, 0.17, 0.11, 0.06, 0.2],
              [0.26, 0.18, 0.12, 0.08, 0.2]]},
]

```

Figure 7.39 Rapid Runoff Parameters File YAML example

The indentations shown in the input YAML file are for readability purposes only, the file will be read without indentations as long as all Python data structures are correctly represented.

It may be helpful for users less familiar with Python data structures to set up an excel based matrix or similar such as that shown in Figure 7.40. The cross-tab style matrix clearly shows which runoff coefficient is associated with which RI and SMD class. A set of concatenate excel worksheet functions may then be developed to convert the matrix into the required Python data structure input.

		Antecedent Soil Moisture Deficit			
		<5	5-10	10-15	15-999
Rainfall Intensity (mm)	Class	5	10	15	999
<20	20	23.1%	16.5%	13.2%	10.6%
20-40	40	42.4%	30.3%	24.2%	19.4%
40-80	80	73.2%	52.3%	41.8%	33.4%
80-999	999	98.0%	74.3%	59.4%	47.5%

Figure 7.40 Excel based Rapid Runoff Coefficient Matrix

7.5.4 Runoff recharge with downstream routing

Four input files are required for this process:

- A spatial mapping file (swrecharge_zone_mapping)
- A zone name file (swrecharge_zone_names)
- Parameterise a runoff recharge proportion for each zone (swrecharge_proportion)
- Parameterise a runoff recharge limit for each zone (swrecharge_limit)

The spatial mapping file keyword is swrecharge_zone_mapping, the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.41. Column A lists the node numbers, and Column B lists the runoff recharge mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	2
7	7	2
8	8	2
9	9	2
10	10	2
11		

Figure 7.41 Runoff Recharge Spatial Mapping CSV example

In the case of a YAML file, the format is shown in Figure 7.42. The first line of the YAML file contains the keyword swrecharge_zone_mapping followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the runoff recharge mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
swrecharge_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 2
  6: 2
  7: 2
  8: 2
  9: 2
```

Figure 7.42 Runoff Recharge Spatial Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The runoff recharge zone name file keyword is `swrecharge_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.43. The first line of the YAML file contains the keyword `swrecharge_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
swrecharge_zone_names:
  1: Catchment1
  2: Catchment2
```

Figure 7.43 Runoff Recharge Zone Name File YAML example

The proportion of available water assigned to runoff recharge is represented by the keyword `swrecharge_proportion`. The keyword pair is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.44. The file contains twelve rows and multiple columns. The rows represent months of the year and are specified by integers in column 1. Each of the following columns represents a runoff recharge zone, in the numeric order specified by the macropore zone name file, starting with zone 1. The monthly proportions for each zone are represented by float values of less than 1.0 in each of these columns.

	A	B	C
1	1	0.1	0.1
2	2	0.1	0.1
3	3	0.1	0.1
4	4	0.1	0.1
5	5	0.1	0.1
6	6	0.1	0.1
7	7	0.1	0.1
8	8	0.1	0.1
9	9	0.1	0.1
10	10	0.1	0.1
11	11	0.1	0.1
12	12	0.1	0.1

Figure 7.44 Runoff recharge proportion CSV example

In the case of a YAML file the format is shown in Figure 7.45. Comments are added for readability as indicated by a preceding `#` sign. The first machine readable line of the YAML file contains the keyword `swrecharge_proportion` followed by a colon. The following lines have an obligatory indentation, followed by the month number with a colon, another obligatory space, and a list of proportions represented by float values of less than 1.0. The order of items in the list follows the numeric order of zone numbers specified in the name file. The indentation and space obligations are required to preserve the Python dictionary data structure.

```
# Columns are zones
swrecharge_proportion:
  1: [0.1, 0.0] # January
  2: [0.1, 0.0] # February
  3: [0.1, 0.0] # March
  4: [0.1, 0.0] # April
  5: [0.1, 0.0] # May
  6: [0.1, 0.0] # June
  7: [0.1, 0.0] # July
  8: [0.1, 0.0] # August
  9: [0.1, 0.0] # September
  10: [0.1, 0.0] # October
  11: [0.1, 0.0] # November
  12: [0.1, 0.0] # December
```

Figure 7.45 Runoff recharge proportion YAML example

The limit threshold of available water assigned to runoff recharge is represented by the keyword `swrecharge_limit`. The keyword pair is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.46. The file contains twelve rows and multiple columns. The rows represent months of the year and are specified by integers in column 1. Each of the following columns represents a runoff recharge limit, in the numeric order specified by the macropore zone name file, starting with zone 1. The limits for each zone are expressed in millimetres represented by float values (integers are not accepted) in each of these columns.

	A	B	C
1	1	2	2
2	2	2	2
3	3	2	2
4	4	2	2
5	5	2	2
6	6	2	2
7	7	2	2
8	8	2	2
9	9	2	2
10	10	2	2
11	11	2	2
12	12	2	2

Figure 7.46 Runoff recharge limit CSV example

In the case of a YAML file the format is shown in Figure 7.47. Comments are added for readability as indicated by a preceding `#` sign. The first machine readable line of the YAML file contains the keyword `swrecharge_limit` followed by a colon. The following lines have an obligatory indentation, followed by the month number with a colon, another obligatory space, and a list of limit thresholds represented by float values in millimetres. The order of items in the list follows the numeric order of zone numbers specified in the name file. The indentation and space obligations are required to preserve the Python dictionary data structure.

```
# Columns are recharge zones
swrecharge_limit:
  1: [0.0, 2.0] # January
  2: [0.0, 2.0] # February
  3: [0.0, 2.0] # March
  4: [0.0, 2.0] # April
  5: [0.0, 2.0] # May
  6: [0.0, 2.0] # June
  7: [0.0, 2.0] # July
  8: [0.0, 2.0] # August
  9: [0.0, 2.0] # September
  10: [0.0, 2.0] # October
  11: [0.0, 2.0] # November
  12: [0.0, 2.0] # December
```

Figure 7.47 Runoff recharge limit YAML example

7.5.5 Single cell runoff recharge without downstream routing

This process follows a similar input format as its routed counterpart (see Section 7.5.4).

This process is a legacy feature that is no longer being maintained.

7.5.6 Macropore recharge

Seven inputs are required for this process:

- A master file switch for macropore occurrence (macropore_activation_option)
- A spatial mapping file of macropore zones (macropore_zone_mapping)
- A zone name file (macropore_zone_names)
- Parameterise a macropore proportion for each zone (macropore_proportion)
- Parameterise a macropore limit for each zone (macropore_limit)
- Parameterise a macropore activation for each zone (macropore_activation)
- Parameterise a macropore recharge for each zone (macropore_recharge)

The activation option switch has the keyword macropore_activation_option, the keyword pair is written directly into the master control file as text. If macropore activation is based on the soil moisture deficit, then “SMD” is written. If macropore activation is based on rainfall intensity, the “RI” is written. If left, blank then the program will default to the SMD option.

The spatial mapping file keyword is macropore_zone_mapping, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.48. Column A lists the node numbers. Column B lists the rapid runoff mapping zone to which the node is assigned. Both column A and column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	2
7	7	2
8	8	2
9	9	2
10	10	2

Figure 7.48 Rapid Runoff Zone Mapping CSV example

In the case of a YAML file the format is shown in Figure 7.49. The first line of the YAML file contains the keyword `macropore_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the macropore mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
macropore_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 2
  6: 2
  7: 2
  8: 2
  9: 2
```

Figure 7.49 Rapid Runoff Zone Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The macropore zone name file keyword is `macropore_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.50. The first line of the YAML file contains the keyword `macropore_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
macropore_zone_names:
  1: Catchment1
  2: Catchment2
```

Figure 7.50 Macropore Zone Name File YAML example

The proportion of available water assigned to macropore is represented by the keyword `macropore_proportion`. The keyword pair is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.50. The file contains twelve rows and multiple columns. The rows represent months of the year and are specified by integers in column 1. Each of the following columns

represents a macropore zone, in the numeric order specified by the macropore zone name file, starting with zone 1. The monthly proportions for each zone are represented by float values of less than 1.0 in each of these columns.

	A	B	C
1	1	0.05	0.05
2	2	0.05	0.05
3	3	0.05	0.05
4	4	0.05	0.05
5	5	0.05	0.05
6	6	0.05	0.05
7	7	0.05	0.05
8	8	0.05	0.05
9	9	0.05	0.05
10	10	0.05	0.05
11	11	0.05	0.05
12	12	0.05	0.05

Figure 7.51 Macropore proportion CSV example

In the case of a YAML file the format is shown in Figure 7.52. Comments are added for readability as indicated by a preceding # sign. The first machine readable line of the YAML file contains the keyword `macropore_proportion` followed by a colon. The following lines have an obligatory indentation, followed by the month number with a colon, another obligatory space, and a list of proportions represented by float values of less than 1.0. The order of items in the list follows the numeric order of zone numbers specified in the name file. The indentation and space obligations are required to preserve the Python dictionary data structure.

```
# Columns are macropore zones
macropore_proportion:
  1: [0.05, 0.05] # January
  2: [0.05, 0.05] # February
  3: [0.05, 0.05] # March
  4: [0.05, 0.05] # April
  5: [0.05, 0.05] # May
  6: [0.05, 0.05] # June
  7: [0.05, 0.05] # July
  8: [0.05, 0.05] # August
  9: [0.05, 0.05] # September
  10: [0.05, 0.05] # October
  11: [0.05, 0.05] # November
  12: [0.05, 0.05] # December
```

Figure 7.52 Macropore proportion YAML example

The shapes and structure of both the CSV and YAML options described for the macropore proportion parameter input is replicated for the input of macropore limit, macropore activation, and macropore recharge parameters. The main difference is that the macropore limit and activation unit values are millimetres expressed in float format (integers are not accepted), whilst the proportion and recharge values are decimal fractions that cannot exceed 1.0. Below is a summary table of input units per interflow mechanism.

Mechanism	Input units	Number Format
Macropore Proportion	Decimal proportion	Float less than 1.0
Macropore Limit	Millimetres	Float
Macropore Activation	Millimetres	Float

Macropore Recharge	Decimal Proportion	Float less than 1.0
--------------------	--------------------	---------------------

7.5.7 Soil moisture accounting and actual evapotranspiration calculation

Eleven inputs are listed for this process, although the actual number required varies depending on the methodology used:

- A control switch for input method (fao_input)
- A spatial mapping file of land use zones (lu_spatial)
- A land use zone name file (landuse_zone_names)
- A spatial mapping file of soil zones (soil_spatial)
- A soil zone name file (soil_zone_names)
- Wilting point – WP, field capacity - FC, and depletion fraction - P data (soil_static_params)
- Rooting depths - ZR (zr)
- Starting soil moisture deficit - SMD (smd)
- Crop coefficients - Kc (kc)
- Total available total evaporable water - TAW (taw)
- Readily available readily evaporable water - RAW (raw)

The most widely used input method is to base soil moisture calculations on a single spatial distribution, commonly land use. In this case the control switch keyword fao_input takes the keyword pair “l”. Using this option RAW and TAW values are read directly (from the raw and taw keyword pairs) and not calculated from soil parameters within the soil_static_params keyword.

The spatial mapping file keyword is lu_spatial, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.53. Column A lists the node numbers. Columns B onwards represent each of the land use zones in the same order as those listed in the land use zone name file. In the example below, there are four land use zones represented by Columns B through E. Each node can be represented by a single land use or as a proportion of several different land use types. In the case of a single land use, the representative column is given a value of 1, and all other land use zones are assigned 0. This is the case for node number 1 shown in the example in Figure 7.53. In the case of multiple land use zones per model node, the relative proportions are entered into each column. In the example, Nodes 2 through 10 are each made up by 25% of all four land use types. Column A inputs should be integers, columns B through E inputs can be integers or floats, each row should add up to 1 within a margin of error of 1e-5.

	A	B	C	D	E
1	1	0	1	0	0
2	2	0.25	0.25	0.25	0.25
3	3	0.25	0.25	0.25	0.25
4	4	0.25	0.25	0.25	0.25
5	5	0.25	0.25	0.25	0.25
6	6	0.25	0.25	0.25	0.25
7	7	0.25	0.25	0.25	0.25
8	8	0.25	0.25	0.25	0.25
9	9	0.25	0.25	0.25	0.25
10	10	0.25	0.25	0.25	0.25

Figure 7.53 Land use Zone Mapping CSV example

In the case of a YAML file the format is shown in Figure 7.54. Comments are added for readability as indicated by a preceding # sign. The first machine readable line of the YAML file contains the keyword `lu_spatial` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the land use zone proportion to which the node is assigned, either as an integer or a float. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# Columns are land use zones proportions
lu_spatial:
  1: [0, 1, 0, 0]
  2: [0.25, 0.25, 0.25, 0.25]
  3: [0.25, 0.25, 0.25, 0.25]
  4: [0.25, 0.25, 0.25, 0.25]
  5: [0.25, 0.25, 0.25, 0.25]
  7: [0.25, 0.25, 0.25, 0.25]
  8: [0.25, 0.25, 0.25, 0.25]
  10: [0.25, 0.25, 0.25, 0.25]
  6: [0.25, 0.25, 0.25, 0.25]
  9: [0.25, 0.25, 0.25, 0.25]
```

Figure 7.54 Land use Zone Mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential, as long as all nodes are included and the total number of nodes matches the `num_nodes` keyword pair specification.

7.5.8 Percolation rejection

This process follows the same spatial mapping described for the Soil Moisture Accounting and AE calculation process (see Section 7.5.7), as defined by the `landuse_zone_names` and `lu_spatial` keyword mappings. One extra input file is also required for this process, along with an activation parameter:

- Specify whether the percolation limit threshold will be defined by a timeseries (`percolation_rejection_use_timeseries`)
- Parameterise a percolation limit threshold for each zone (`percolation_rejection`)

The `percolation_rejection_use_timeseries` keyword pair defines whether the percolation limit threshold will be defined using a timeseries or not. Accepted values can be either “true” or “false”. If set to “true”, then the provided timeseries file follows the format described for climatic timeseries in Section 7.4.

The `percolation_rejection` keyword pair is a link to a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.55. Column A contains the keyword 'percolation_rejection', and each of the following columns represents a zone, in the numeric order they were specified by the landuse zone name file. The percolation limit thresholds are represented in millimetres by floats in each of these columns.

	A	B	C	D	E
1	'percolation_rejection'	10	10	10	10

Figure 7.55 Percolation rejection limit CSV file example

In the case of a YAML file, the format is shown in Figure 7.56. Comments are added for readability as indicated by a preceding # sign. The first machine readable line of the YAML file contains the keyword `percolation_rejection` followed by a colon. The following line has an obligatory indentation, followed by the keyword `percolation_rejection` with a colon, another obligatory space, and a list of percolation limit thresholds in millimetres represented by floats. The order of items in the list follows the numeric order of zone numbers specified in the zone name file. The indentation and space obligations are required to preserve the Python dictionary data structure.

```
# Columns are zones
percolation_rejection:
  percolation_rejection: [10.0, 10.0, 10.0, 10.0]
```

Figure 7.56 Percolation rejection limit YAML file example

7.5.9 Leakage below the root zone

Four files are required for this process:

- A spatial mapping file (`subroot_zone_mapping`)
- A zone name file (`subroot_zone_names`)
- Parameterise a permitted leakage proportion for each zone (`subroot_leakage_fraction`)
- A maximum leakage rate timeseries (`subroot_leakage_ts`)

The spatial mapping file keyword is `subroot_zone_mapping`, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.57. Column A lists the node numbers. Column B lists the leakage mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers. Column C lists the proportion of the node surface area that leakage occurs as a float lower than 1.0.

	A	B	C
1	1	1	1
2	2	1	1
3	3	1	1
4	4	1	1
5	5	2	1
6	6	2	1
7	7	2	1
8	8	2	1
9	9	2	1
10	10	2	1
11			

Figure 7.57 Subroot leakage zone mapping file CSV example

In the case of a YAML file the format is shown in Figure 7.58. The first line of the YAML file contains the keyword `subroot_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and a list of parameters represented as floats. The order of the parameters within the list follows the order described earlier, i.e. the subroot leakage mapping zone to which the node is assigned as an integer, followed by the proportion as a float lower than 1.0. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# Columns are
# 1) zone
# 2) proportion
subroot_zone_mapping:
  1: [1, 1.0]
  2: [1, 1.0]
  3: [1, 1.0]
  4: [1, 1.0]
  5: [2, 1.0]
  10: [2, 1.0]
  6: [2, 1.0]
  7: [2, 1.0]
  8: [2, 1.0]
  9: [2, 1.0]
```

Figure 7.58 Subroot leakage zone mapping file YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The subroot leakage zone name file keyword is `subroot_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.38. The first line of the YAML file contains the keyword `subroot_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# subroot zone names.
subroot_zone_names:
  1: Catchment1
  2: Catchment2
```

Figure 7.59 Subroot leakage zone name file YAML example

The proportion of subroot leakage to permit is represented by the keyword `subroot_leakage_fraction`. The keyword pair is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.60. Column A lists the node numbers as integers, and Column B lists the permitted leakage proportion as floats lower than 1.0.

	A	B
1	1	0.5
2	2	0.5
3	3	0.5
4	4	0.5
5	5	0.5
6	6	0.5
7	7	0.5
8	8	0.5
9	9	0.5
10	10	0.5
11		

Figure 7.60. Subroot leakage proportion CSV example

In the case of a YAML file the format is shown in Figure 7.61. The first line of the YAML file contains the keyword `subroot_leakage_fraction` followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and leakage proportion as floats lower than 1.0. The indentation and space obligations are to preserve the Python dictionary data structure.

```
subroot_leakage_fraction:
  1: 0.5
  2: 0.5
  4: 0.5
  5: 0.5
  6: 0.5
  7: 0.5
  8: 0.5
  9: 0.5
  10: 0.5
  3: 0.5
```

Figure 7.61. Subroot leakage proportion YAML example

The maximum leakage rate is specified using the keyword `subroot_leakage_ts`. The keyword pair is a link to a leakage timeseries file, that follows the format described for climatic timeseries files in Section 7.3.

7.5.10 Interflow

Six input files and two activation parameters are required for this process:

- A spatial mapping file (interflow_zone_mapping)
- A zone name file (interflow_zone_names)
- Parameterise an initial interflow store for each zone (init_interflow_store)
- Parameterise in interflow store bypass proportion for each zone (interflow_store_bypass)
- Parameterise an interflow infiltration limit threshold for each zone (infiltration_limit)
- Parameterise an interflow decay proportion for each zone (infiltration_decay)
- Specify whether the interflow infiltration limit threshold will be obtained from a timeseries (infiltration_limit_use_timeseries)
- Specify whether the interflow decay will be obtained from a timeseries (infiltration_decay_use_timeseries)

The spatial mapping file keyword is `interflow_zone_mapping`, the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.62. Column A lists the node numbers, and Column B lists the interflow mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	2
7	7	3
8	8	2
9	9	2
10	10	2

Figure 7.62 Interflow zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.63. The first line of the YAML file contains the keyword `interflow_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the interflow mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
interflow_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 2
  6: 2
  7: 3
  8: 2
  9: 2
```

Figure 7.63 Interflow zone mapping YAML example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the num_nodes keyword pair specification.

The interflow zone name file keyword is interflow_zone_names, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.64. The first line of the YAML file contains the keyword interflow_zone_names followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
interflow_zone_names:
  1: Catchment1
  2: Catchment2
  3: somethingelse
```

Figure 7.64 Interflow zone name file YAML example

The initial interflow store for each zone is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.65. Column A list the zone numbers as integers, and Column B lists the initial interflow store in millimetres as floats.

	A	B
1	1	10
2	2	10
3	3	10
4		

Figure 7.65 Initial interflow store CSV example

In the case of a YAML file the format is shown in Figure 7.66. The first line of the YAML file contains the keyword init_interflow_store followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and the initial interflow store in millimetres as floats. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# integers are interflow zones
init_interflow_store:
  1: 10.0
  2: 10.0
  3: 10.0
```

Figure 7.66 Initial interflow store YAML example

The interflow store bypass proportion for each zone is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.67. Column A list the zone numbers as integers, and Column B lists the interflow store bypass proportion as floats lower than 1.0.

	A	B
1	1	0.05
2	2	0.05
3	3	0.05

Figure 7.67 Interflow store bypass CSV example

In the case of a YAML file the format is shown in Figure 7.68. The first line of the YAML file contains the keyword `interflow_store_bypass` followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and the initial interflow store bypass proportion as floats lower than 1.0. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# integers are interflow zones
interflow_store_bypass:
  1: 0.05
  2: 0.05
  3: 0.05
```

Figure 7.68 Interflow store bypass YAML example

The interflow infiltration limit threshold for each zone is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.69. Column A list the zone numbers as integers, and Column B lists the interflow infiltration limit in millimetres as floats.

	A	B
1	1	0.7
2	2	0.7
3	3	0.7

Figure 7.69 Interflow infiltration limit CSV example

In the case of a YAML file the format is shown in Figure 7.70. The first line of the YAML file contains the keyword `infiltration_limit` followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and the interflow infiltration limit in millimetres as floats. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# integers are interflow zones
infiltration_limit:
  1: 0.7
  2: 0.7
  3: 0.7
```

Figure 7.70 Interflow infiltration limit YAML example

The interflow decay for each zone is a link to a CSV or YAML file. In the case of a CSV file, the format is shown in Figure 7.71. Column A list the zone numbers as integers, and Column B lists the interflow decay proportion as floats lower than 1.0.

	A	B
1	1	0.05
2	2	0.05
3	3	0.05

Figure 7.71 Interflow decay proportion CSV example

In the case of a YAML file the format is shown in Figure 7.72. The first line of the YAML file contains the keyword `infiltration_decay` followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and the interflow decay proportion as floats lower than 1.0. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# integers are interflow zones
infiltration_decay:
  1: 0.05
  2: 0.05
  3: 0.05
```

Figure 7.72 Interflow decay proportion YAML example

The `infiltration_limit_use_timeseries` keyword pair defines whether the interflow infiltration limit threshold will be defined using a timeseries or not. Accepted values can be either “true” or “false”. If set to “true”, then the provided timeseries file follows the format described for climatic timeseries in Section 7.4.

The `infiltration_decay_use_timeseries` keyword pair defines whether the interflow decay will be defined using a timeseries or not. Accepted values can be either “true” or “false”. If set to “true”, then the provided timeseries file follows the format described for climatic timeseries in Section 7.4.

7.5.11 Attenuation of recharge

One input file is required for this process:

- Parameterise the recharge attenuation process parameters (`recharge_attenuation_params`)

The recharge attenuation parameters keyword is `recharge_attenuation_params`, the keyword pair is a link to a CSV or a YAML file. In the case of a CSV, the format is shown in Figure 7.73. Column A list the SWAc node numbers as integers, Column B lists the initial recharge store in millimetres as floats, Column C lists the proportion of recharge store to be released each time step as floats less than 1.0, and Column D lists the maximum rate of release in millimetres as floats.

	A	B	C	D
1	1	1	0.05	2
2	2	1	0.05	2
3	3	1	0.05	2
4	4	1	0.05	2
5	5	1	0.05	2
6	6	1	0.05	2
7	7	1	0.05	2
8	8	1	0.05	2
9	9	1	0.05	2
10	10	1	0.05	2
11				

Figure 7.73 Recharge attenuation parameters CSV file example

In the case of a YAML file, the format is shown in Figure 7.74. Comments are added for readability as indicated by a preceding # sign. The first machine readable line of the YAML file contains the keyword `recharge_attenuation_params` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and a list of parameters represented as floats. The order of the parameters within the list follows the order described earlier, i.e. the first parameter is the initial recharge store in millimetres, the second parameter is the recharge store release proportion lower than 1.0, and the third parameter is the maximum rate of release in millimetres

```
# Columns are
# 1) init_recharge_store
# 2) release_proportion
# 3) release_limit
recharge_attenuation_params:
  1: [1.0, 0.05, 2.0]
  2: [1.0, 0.05, 2.0]
  4: [1.0, 0.05, 2.0]
  5: [1.0, 0.05, 2.0]
  6: [1.0, 0.05, 2.0]
  7: [1.0, 0.05, 2.0]
  8: [1.0, 0.05, 2.0]
  9: [1.0, 0.05, 2.0]
  10: [1.0, 0.05, 2.0]
  3: [1.0, 0.05, 2.0]
```

Figure 7.74 Recharge attenuation parameters YAML file example

7.5.12 Solute transport

This process includes two components that can be activated:

- Inclusion of historical release of solutes, so that solute mass already in the unsaturated zone can be represented (as opposed to a “clean” initial condition) (`historical_solute_process`).
- Modelling of solute transport during the current model period (`solute_process`).

The historical solute process requires three additional keyword pairs:

- An array of solute mass released in each cell during each time period of the preceding simulation (historical_mi_array_kg_per_time_period)
- The starting date of the preceding simulation (historical_start_date)
- The time periods over which results were aggregated in the preceding simulation (historical_time_periods)

The start date of the preceding model is assigned with the required keyword `historical_start_date`. The pair to this keyword has to be provided as text in the format YYYY-MM-DD.

The length of each historical time period is assigned via the required keyword `historical_time_periods`. The pair to this keyword is a link to a CSV file. The time period file should contain two columns with no headers as shown in Figure 7.76, and each row represents a time period. Column A represents the start of each time period in days from the start date, and Column B represents the end of each period in days from the start date. In the example shown in Figure 7.76, the model has three periods. Period 1 consists of 31 days (days 1-31), period 2 contains 28 days (days 31-59), and period 3 contains 31 days (days 59-90). This is the typical setup for running SWAc on monthly time periods.

	A	B
1	1	31
2	31	59
3	59	90

Figure 7.75 Historical time period CSV input example

The primary solute process requires seven additional keyword pairs:

- Parameterise the logarithm base (solute_calibration_a)
- Parameterise the mean (solute_calibration_mu)
- Parameterise the standard deviation (solute_calibration_sigma)
- Parameterise the coefficient α (solute_calibration_alpha)
- Parameterise the effective porosity (solute_calibration_effective_porosity)
- Parameterise the average depth to groundwater (solute_depth_to_water)
- Parameterise the solute loading (solute_loading)

The logarithm base for the CDF shown in Equation (6.6) keyword is `solute_calibration_a`. The keyword pair is a float defined directly in the master control file.

The keyword giving the mean of the variable's logarithm for the CDF in Equation (6.6) is `solute_calibration_mu`. The keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.76. Column A lists the SWAc node numbers as integers, and Column B lists the means as floats.

	A	B
1	1	1.58
2	2	1.58
3	3	1.58
4	4	1.88
5	5	2.42
6	6	1.62
7	7	1.11
8	8	1.58
9	9	1.58
10	10	1.58
11		

Figure 7.76 Solute mean CSV file example

In the case of a YAML file, the format is shown in Figure 7.77. The first line of the YAML file contains the keyword `solute_calibration_mu` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the mean value for that node as a float. The indentation and space obligations are to preserve the Python dictionary data structure.

```
solute_calibration_mu:
  1: 1.58
  2: 1.58
  4: 1.58
  5: 1.58
  6: 1.58
  7: 1.58
  8: 1.58
  9: 1.58
  10: 1.58
  3: 1.58
```

Figure 7.77 Solute mean YAML file example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The standard deviation of the variable's logarithm for the CDF shown in Equation (6.6) keyword is `solute_calibration_sigma`. The keyword pair is a float defined directly in the master control file.

The coefficient α for the CDF shown in Equation (6.6) keyword is `solute_calibration_alpha`. The keyword pair is a float defined directly in the master control file.

The effective porosity for the CDF shown in Equation (6.6) keyword is `solute_calibration_effective_porosity`. The keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.78. Column A lists the SWAc node numbers as integers, and Column B lists the effective porosity as floats.

	A	B
1	1	0.15
2	2	0.15
3	3	0.12
4	4	0.04
5	5	0.18
6	6	0.15
7	7	0.15
8	8	0.23
9	9	0.15
10	10	0.01

Figure 7.78 Effective porosity CSV file example

In the case of a YAML file, the format is shown in Figure 7.79. The first line of the YAML file contains the keyword `solute_calibration_effective_porosity` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the effective porosity for that node as a float. The indentation and space obligations are to preserve the Python dictionary data structure.

```
solute_calibration_effective_porosity:
  1: 0.15
  2: 0.15
  4: 0.04
  5: 0.18
  6: 0.15
  7: 0.15
  8: 0.23
  9: 0.15
  10: 0.01
  3: 0.12
```

Figure 7.79 Effective porosity YAML file example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The average depth to groundwater for the CDF shown in Equation (6.6) keyword is `solute_depth_to_water`. The keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.80. Column A lists the SWAc node numbers as integers, and Column B lists the depth to groundwater in metres as floats.

	A	B
1	1	5
2	2	24
3	3	18
4	4	7
5	5	14
6	6	51
7	7	3
8	8	2
9	9	17
10	10	22

Figure 7.80 Average depth to groundwater CSV file example

In the case of a YAML file, the format is shown in Figure 7.81. The first line of the YAML file contains the keyword `solute_depth_to_water` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the average depth to groundwater for that node in metres as a float. The indentation and space obligations are to preserve the Python dictionary data structure.

```
solute_depth_to_water:
  1: 5
  2: 24
  4: 7
  5: 14
  6: 51
  7: 3
  8: 2
  9: 17
  10: 22
  3: 18
```

Figure 7.81 Average depth to groundwater YAML file example

Note in both the CSV and YAML case, the order of nodes does not need to be sequential as long as all nodes are included, and the total number of nodes matches the `num_nodes` keyword pair specification.

The solute loading keyword is `solute_loading`. The keyword pair is a link to a CSV file and the format is shown in Figure 7.82. The CSV file must be formatted as follows:

- Column A lists the SWAc node numbers (integer);
- Column B lists the UNIQUE numbers (not used within SWAc);
- Column C lists the X coordinate (not used within SWAc);
- Column D lists the Y coordinate (not used within SWAc);
- Column E lists the maximum solute loading in kilograms per hectare (floating point value);
- Column F lists the HER at 5% in millimetres per day (floating point value);

- Column G lists the HER at 50% in millimetres per day (floating point value);
- Column H lists the HER at 95% in millimetres per day (floating point value);
- Column I lists the 5% load (not used within SWAc);
- Column J lists the 50% load (not used within SWAc); and
- Column K lists the 95% load (not used within SWAc).

	A	B	C	D	E	F	G	H	I	J	K
1	1	156132	200000	118000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
2	2	156132	200000	119000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
3	3	156132	200000	120000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
4	4	156132	200000	121000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
5	5	156132	200000	122000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
6	6	156132	200000	123000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
7	7	156132	200000	124000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
8	8	156132	200000	125000	144.5687	11.75727	146.4513	351.6891	7.228434	72.28434	137.3403
9	9	156132	200000	126000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506
10	10	156132	200000	127000	361.4217	11.75727	146.4513	351.6891	18.07109	180.7109	343.3506

Figure 7.82 Solute loading CSV file example

7.5.13 Surface water release

One input file is required for this process:

- Parameterise the surface water release parameters (sw_params)

The surface water release parameters keyword is sw_params, the keyword pair is a link to a CSV or a YAML file. In the case of a CSV, the format is shown Figure 7.83. Column A list the SWAc node numbers as integers, Column B lists the initial surface water store in millimetres as floats, and Column C lists the proportion of surface water store to be released each time step as floats less than 1.0.

	A	B	C
1	1	1	0.7
2	2	1	0.7
3	3	1	0.7
4	4	1	0.7
5	5	1	0.7
6	6	1	0.7
7	7	1	0.7
8	8	1	0.7
9	9	1	0.7
10	10	1	0.7
11			

Figure 7.83 Surface water release parameters CSV example

In the case of a YAML file, the format is shown in Figure 7.84. Comments are added for readability as indicated by a preceding # sign. The first machine readable line of the YAML file contains the keyword sw_params followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and a list of parameters represented as floats. The order of the parameters within the list follows the order described earlier, i.e. the first parameter

is the initial surface water store in millimetres (as a floating point value), and the second parameter is the surface water release proportion (as a floating point value from 0.0 to 1.0).

```
# Columns are
# 1) initial_sw_store
# 2) release_proportion
sw_params:
  1: [1.0, 0.7]
  2: [1.0, 0.7]
  4: [1.0, 0.7]
  5: [1.0, 0.7]
  6: [1.0, 0.7]
  7: [1.0, 0.7]
  8: [1.0, 0.7]
  9: [1.0, 0.7]
  10: [1.0, 0.7]
  3: [1.0, 0.7]
```

Figure 7.84 Surface water release parameters YAML example

7.5.14 Surface water ponding

This process requires the following 10 inputs:

- A spatial mapping file (sw_zone_mapping)
- A zone name file (sw_zone_names)
- A proportion of water released downstream from ponded features for each month and each zone (sw_downstream)
- A minimum water depth for each month and each zone before bed infiltration can occur from the ponded feature (sw_activation)
- The proportion of ponded water infiltrating through the bed to the recharge attenuation store (sw_bed_infiltration)
- The proportion of ponded water passing through the bed and bypassing the recharge attenuation store (sw_direct_recharge)
- A monthly open water evaporation coefficient for each zone (sw_pe_to_open_water)
- An initial ponding depth for all features (sw_init_ponding)
- A maximum ponding depth for all features (sw_max_ponding)
- The proportion of each cell subject to ponding (sw_ponding_area)

The spatial mapping file keyword is sw_zone_mapping, the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.85. Column A lists the node numbers. Column B lists the surface water mapping zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	3
7	7	3
8	8	3
9	9	4
10	10	4

Figure 7.85 Surface water zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.86. The first line of the YAML file contains the keyword `sw_zone_mapping` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the rapid runoff mapping zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
sw_zone_mapping:
  1: 1
  2: 1
  3: 1
  4: 2
  5: 2
  10: 4
  6: 3
  7: 3
  8: 3
  9: 4
```

Figure 7.86 Surface water zone mapping YAML example

The surface water ponding zone name file keyword is `sw_zone_names`, the keyword pair is a link to a YAML file. The YAML file format is shown in Figure 7.87. The first line of the YAML file contains the keyword `sw_zone_names` followed by a colon. The following lines have an obligatory indentation, followed by the zone number with a colon, another obligatory space, and the arbitrary zone name. It is recommended the user inputs a meaningful name here that will inform readers. The indentation and space obligations are to preserve the Python dictionary data structure.

```
sw_zone_names:
  1: Catchment1
  2: Catchment2
  3: Somethingelse
```

Figure 7.87 Surface water ponding zone name file YAML example

The keyword for assigning the proportion of flow released each day from ponded features is in each zone is `sw_downstream`; the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.88. The file contains twelve rows and multiple columns. The rows represent months of the year and are specified by integers in column 1. Each of the following columns represents a surface water ponding zone, in the numeric order specified by the surface water zone

name file, starting with zone 1. The proportions released each day for every calendar month and each zone are represented by floating point values of up to 1.0 in each of these columns.

	A	B	C	D
1	1	0.6	0.9	1.0
2	2	0.6	0.9	1.0
3	3	0.5	0.85	1.0
4	4	0.4	0.8	1.0
5	5	0.35	0.7	1.0
6	6	0.25	0.6	1.0
7	7	0.2	0.6	1.0
8	8	0.15	0.6	1.0
9	9	0.2	0.7	1.0
10	10	0.4	0.8	1.0
11	11	0.5	0.85	1.0
12	12	0.6	0.9	1.0

Figure 7.88 Surface water ponding downstream release CSV example

The first machine readable line of the YAML file contains the keyword `sfr_monthly_flow_proportions` followed by a colon, as shown in Figure 7.89. The following lines have an obligatory indentation, followed by the month number with a colon, another obligatory space, and a list of proportions represented by floating point values of up to 1.0. The order of items in the list follows the numeric order of zone numbers specified in the name file. The indentation and space obligations are required to preserve the Python dictionary. data structure.

```
# Columns are surface water ponding zones, rows are months
sfr_monthly_flow_proportions:
  1: [0.6,0.9,1.0] # January
  2: [0.6,0.9,1.0] # February
  3: [0.5,0.85,1.0] # March
  4: [0.4,0.8,1.0] # April
  5: [0.35,0.7,1.0] # May
  6: [0.25,0.6,1.0] # June
  7: [0.2,0.6,1.0] # July
  8: [0.15,0.6,1.0] # August
  9: [0.2,0.7,1.0] # September
  10: [0.4,0.8,1.0] # October
  11: [0.5,0.85,1.0] # November
  12: [0.6,0.9,1.0] # December
```

Figure 7.89 SFR monthly flow proportions YAML example

The input files for `sw_activation`, `sw_bed_infiltration`, `sw_direct_recharge` and `sw_pe_to_open_water` follow the same format as the YAML and CSV examples shown for `sfr_monthly_flow_proportions`, with corresponding keywords for YAML files. In each case, after the month number, subsequent columns contain floating point values for corresponding surface water zones. Values for `sw_activation` are in mm; `sw_bed_infiltration` and `sw_direct_recharge` are proportions ranging from 0.0 to 1.0; and `sw_pe_to_open_water` are coefficients which may take any user-defined value.

The keyword pairs for `sw_init_ponding` and `sw_max_ponding` are each entered into the main control file as a single, floating point value in mm.

For the area subject to ponding (sw_ponding_area), the keyword pair is a link to either a CSV file or a YAML file. In the case of a CSV file the format is shown in Figure 7.90. Column A list the zone numbers as integers, and Column B lists the proportion of cells in that zone subject to ponding as floating point values.

	A	B
1	1	0.01
2	2	0.1
3	3	0.2
4	4	0.3
5	5	0.4
6	6	0.5
7	7	0.6
8	8	0.7
9	9	0.8
10	10	0.9

Figure 7.90 Surface water ponding area CSV example

The format of the YAML file is shown in Figure 7.91. The first machine readable line of the YAML file contains the keyword sw_ponding_area followed by a colon. The following lines have an obligatory indentation, followed by the node number as an integer, followed by a colon and another obligatory space, and the proportion of cells subject to ponding as floating point values. The indentation and space obligations are to preserve the Python dictionary data structure.

```
# SW Release Ponding Area
# integers are zones
sw_ponding_area:
  1: 0.01
  2: 0.1
  3: 0.2
  4: 0.3
  5: 0.4
  6: 0.5
  7: 0.6
  8: 0.7
  9: 0.8
  10: 0.9
```

Figure 7.91 Surface water ponding area YAML example

7.5.15 Surface water routing

This process requires six mandatory inputs, with two optional inputs (depending on whether surface water attenuation is used). These inputs perform the following functions:

- Parameterise the routing network topology and the MODFLOW SFR parameters: (routing_topology)
- Specify whether the model will output a MODFLOW SFR file (output_sfr)
- Specify whether the SFR flows will be subjected to attenuation (attenuate_sfr_flows)
- Assign spatially-varying zones with common flow attenuation properties (sfr_flow_zones)

- Assign monthly proportions of flow release to downstream cells (sfr_flow_monthly_proportions)
- Specify a MODFLOW SFR OBS file (sfr_obs)
- Specify the istcb1 MODFLOW unit number (istcb1)
- Specify the istcb2 MODFLOW unit number (istcb2)

The routing network topology and parameters keyword is routing_topology, the keyword pair is a link to a CSV or a YAML file. In the case of a CSV, the format is shown in Figure 7.92 and described below:

- Column A specifies the SWAc node numbers as an integer value;
- Column B assigns the node number downstream of that specified in Column A, also as an integer;
- Column C specifies whether the node listed in Column A is a designated Stream cell in MODFLOW; accepted values are 1 for Stream cells, or 0 for all other cells not in the Stream network (both must be entered as an integer value);
- Column D specifies the MODFLOW node number that the node corresponds (integer value);
- Column E specifies the MODFLOW SFR parameter RCHLEN, which defines the length of the stream channel within that cell (floating point value);
- Column F specifies the upstream contributing area (floating point value);
- Column G specifies the MODFLOW SFR parameter STRTOP, which defines the top elevation of the stream bed (floating point value);
- Column H specifies the MODFLOW SFR parameter STRTHICK, which defines the thickness of the stream bed (floating point value);
- Column I specifies the MODFLOW SFR parameter STRHC1, which defines the hydraulic conductivity of the stream bed (floating point value);
- Column J specifies the MODFLOW SFR parameter DEPTH2, which defines the average depth of water in the Stream cell (floating point value); and
- Column K specifies the MODFLOW SFR parameter WIDTH2, which defines the average width the stream in this cell (floating point value).

	A	B	C	D	E	F	G	H	I	J	K
1	1	2	0	6	25.679	74.8	23.34	0.4	0.1	0.5	2
2	2	3	0	7	32.0371	82.47	23.9	0.4	1.1	0.5	2
3	3	4	0	8	23.3751	164.64	22.27	0.4	2.1	0.5	2
4	4	5	0	9	16.154	325.08	17.83	0.4	3.1	0.5	2
5	5	6	0	10	9.4946	522.3	14.47	0.4	4.1	0.5	2
6	6	7	1	1	11.656	614.5	13.879	0.4	5.1	0.5	2
7	7	8	1	2	12.958	836.04	13.223	0.4	6.1	0.5	2
8	8	9	1	3	10.4126	999.45	13.247	0.4	7.1	0.5	2
9	9	10	1	4	11.4821	1127.44	13.449	0.4	8.1	0.5	2
10	10	-1	1	5	38.6622	1424.029	23.929	0.4	9.1	0.5	2
11											

Figure 7.92. Routing topology CSV example

In the case of a YAML file, the format is shown in Figure 7.93. The first machine readable line of the YAML file contains the keyword `routing_topology` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and a list of parameters represented as floats. The order of the parameters within the list follows the order described earlier for the CSV example.

```
routing_topology:
  1: [2, 0, 6, 25.679, 74.8, 23.3400002, 0.4, 0.1, 0.5, 2]
  2: [3, 0, 7, 32.0371, 82.47, 23.8999996, 0.4, 1.1, 0.5, 2]
  3: [4, 0, 8, 23.3751, 164.64, 22.2700005, 0.4, 2.1, 0.5, 2]
  4: [5, 0, 9, 16.154, 325.08, 17.8299999, 0.4, 3.1, 0.5, 2]
  5: [6, 0, 10, 9.4946, 522.3, 14.4700003, 0.4, 4.1, 0.5, 2]
  6: [7, 1, 1, 11.656, 614.5, 13.8789997, 0.4, 5.1, 0.5, 2]
  7: [8, 1, 2, 12.958, 836.04, 13.2229996, 0.4, 6.1, 0.5, 2]
  8: [9, 1, 3, 10.4126, 999.45, 13.2469997, 0.4, 7.1, 0.5, 2]
  9: [10, 1, 4, 11.4821, 1127.44, 13.4490004, 0.4, 8.1, 0.5, 2]
  10: [-1, 1, 5, 38.6622, 1424.0285, 23.9290009, 0.4, 9.1, 0.5, 2]
```

Figure 7.93. Routing topology YAML example

The `output_sfr` keyword pair defines whether a MODFLOW SFR file will be exported by SWAc. Accepted values can be either “true” or “false”.

The `attenuate_sfr_flows` keyword pair defines whether the SFR flow will be attenuated with the surface water attenuation process. Accepted values can be either “true” or “false”. If set to “true”, then the keyword pairs `sfr_flow_zones` and `sfr_flow_monthly_proportions` also need to be defined.

The spatial mapping file keyword is `sfr_flow_zones`; the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.62. Column A lists the node numbers, and Column B lists the flow attenuation zone to which the node is assigned. Both Column A and Column B inputs should be integers.

	A	B
1	1	1
2	2	1
3	3	1
4	4	2
5	5	2
6	6	2
7	7	3
8	8	2
9	9	2
10	10	2

Figure 7.94 SFR flow attenuation zone mapping CSV example

In the case of a YAML file the format is shown in Figure 7.95. The first line of the YAML file contains the keyword `sfr_flow_zones` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and a list containing the flow attenuation zone to which the node is assigned as an integer. The indentation and space obligations are to preserve the Python dictionary data structure.

```
sfr_flow_zones:
  1: [1]
  2: [1]
  3: [1]
  4: [2]
  5: [2]
  6: [2]
  7: [3]
  8: [2]
  9: [2]
  10: [2]
```

Figure 7.95 SFR flow attenuation zone mapping YAML example

The keyword for assigning monthly proportions of flow release in each zone is `sfr_flow_monthly_proportions`; the keyword pair is a link to either a CSV or a YAML file. In the case of a CSV file, the format is shown in Figure 7.96. The file contains twelve rows and multiple columns. The rows represent months of the year and are specified by integers in column 1. Each of the following columns represents a flow attenuation zone, in the numeric order specified by the flow attenuation zone name file, starting with zone 1. The monthly proportions for each zone are represented by floating point values of up to 1.0 in each of these columns.

	A	B	C	D
1	1	0.99	1.0	1.0
2	2	0.99	1.0	1.0
3	3	0.99	1.0	1.0
4	4	0.99	1.0	1.0
5	5	0.99	1.0	1.0
6	6	0.98	1.0	1.0
7	7	0.98	0.99	1.0
8	8	0.98	0.99	1.0
9	9	0.99	0.99	1.0
10	10	0.99	1.0	1.0
11	11	0.99	1.0	1.0
12	12	0.99	1.0	1.0

Figure 7.96 SFR monthly flow proportions CSV example

The first machine readable line of the YAML file contains the keyword `sfr_monthly_flow_proportions` followed by a colon. The following lines have an obligatory indentation, followed by the month number with a colon, another obligatory space, and a list of proportions represented by floating point values of up to 1.0. The order of items in the list follows the numeric order of zone numbers specified in the name file. The indentation and space obligations are required to preserve the Python dictionary data structure.

```
# Columns are flow attenuation zones, rows are months
sfr_flow_monthly_proportions:
  1: [0.99,1.0,1.0] # January
  2: [0.99,1.0,1.0] # February
  3: [0.99,1.0,1.0] # March
  4: [0.99,1.0,1.0] # April
  5: [0.99,1.0,1.0] # May
  6: [0.98,1.0,1.0] # June
  7: [0.98,0.99,1.0] # July
  8: [0.98,0.99,1.0] # August
  9: [0.99,0.99,1.0] # September
  10: [0.99,1.0,1.0] # October
  11: [0.99,1.0,1.0] # November
  12: [0.99,1.0,1.0] # December
```

Figure 7.97 SFR monthly flow proportions YAML example

The keyword pair to define a MODFLOW SFR OBS file is `sfr_obs`. The keyword pair is a string defining the name of the OBS file, which is only used for the appropriate creation of the SFR file. SWAc currently lacks the functionality to create an OBS file, and this should be created externally following the MODFLOW documentation

The keyword pair to specify the `istcb1` MODFLOW unit number is `istcb1`. The unit number should be provided as an integer.

The keyword pair to specify the `istcb2` MODFLOW unit number is `istcb2`. The unit number should be provided as an integer.

7.5.16 Surface water discharge

Two input files are required for this process:

- Parameterise the surface water discharge locations (`swdis_locs`)
- Parameterise the surface water discharge frequency (`swdis_f`)

The keyword pair to parameterise the surface water discharge locations is `swdis_locs`, the keyword pair is a link to either a CSV or a YAML file. In the case of CSV, the format is shown in Figure 7.98. Column A list the node number, and Column B lists the index number of discharge from 1 to number of nodes with SW discharges. Both Columns A and B should be integers.

	A	B
1	7	1
2	8	2
3		

Figure 7.98. Surface water discharge locations CSV example

In the case of a YAML file, the format is shown in Figure 7.99. The first machine readable line of the YAML file contains the keyword `swdis_locs` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the index number of discharge from 1 to number of nodes with SW discharges. Both entries should be integers.

```
swdis_locs:
  7: 1
  8: 2
```

Figure 7.99. Surface water discharge locations YAML example

The keyword pair to parameterise the surface water discharge frequency is `swdis_f`. Accepted values for this keyword pair are 0 to specify daily frequency, 1 to specify weekly frequency, and 2 to specify monthly frequency. Inputs to this keyword pair should be integers.

7.5.17 Surface water abstraction

Two input files are required for this process:

- Parameterise the surface water abstraction locations (`swabs_locs`)
- Parameterise the surface water abstraction frequency (`swabs_f`)

The keyword pair to parameterise the surface water abstraction locations is `swabs_locs`, the keyword pair is a link to either a CSV or a YAML file. In the case of CSV, the format is shown in Figure 7.100. Column A list the node number, and Column B lists the index number of abstraction from 1 to number of nodes with SW abstractions. Both Columns A and B should be integers.

	A	B
1	10	1
2		

Figure 7.100. Surface water abstraction locations CSV example

In the case of a YAML file, the format is shown in Figure 7.101. The first machine readable line of the YAML file contains the keyword `swabs_locs` followed by a colon. The following lines have an obligatory indentation, followed by the node number with a colon, another obligatory space, and the index number of abstraction from 1 to number of nodes with SW abstractions. Both entries should be integers.

```
swabs_locs:
  10: 1
```

Figure 7.101. Surface water abstraction locations YAML example

The keyword pair to parameterise the surface water abstraction frequency is `swabs_f`. Accepted values for this keyword pair are 0 to specify daily frequency, 1 to specify weekly frequency, and 2 to specify monthly frequency. Inputs to this keyword pair should be integers.

8 Model output files

8.1 MODFLOW output files

Each of the below files can optionally be enabled for output.

MODFLOW formatted files produced by SWAc include

- Recharge (RCH);
- Evapotranspiration (EVT);
- Segmented Evapotranspiration (ETS);
- Stream (STR); and/or,
- Stream Flow Routing (SFR)

These files are in ASCII format and are described in the MODFLOW input/output manual.

8.2 Breakdown output files

Three file types which are unrelated to MODFLOW can optionally be output to support water balance assessments and model process calibration:

1. Detailed timeseries water balance for a specified node;
2. Detailed timeseries water balance for user defined spatial areas (reporting areas); and/or,
3. Spatial output data for all processes for a specified time

The timeseries files are in identical format, however one is for a singular node and the other outputs the sum of flows for a collection of nodes (typically a surface water catchment upstream of a gauge). These files are comma separated ASCII format output at the temporal resolution of the model and each column corresponds to the input and output of each and every process in the workflow (Figure 5.2). These files can readily be plotted in Microsoft Excel or Golden Software's Grapher. An example is presented in Figure 8.1.

The spatial output files also are ASCII text format, but the rows are the node numbers and the columns are the same input and outputs for each process as discussed above. These files can readily be joined to model mesh shapefiles in Geographic Information System software (GIS).

If the solute transport process is enabled, an additional three CSV files are produced:

1. Solute concentration in recharge reaching the water table in each time period for each model cell, expressed in tons/m³.
2. Solute concentration in runoff and interflow to streams in each time period for each model cell, expressed in tons/m³.
3. A 2D array of solute mass leached in kg in each cell (1 row for each cell), for each time period of the model run (1 column for each time period). The file name is given by the run name assigned to the *run_name* keyword, followed by "*_mi.csv*".

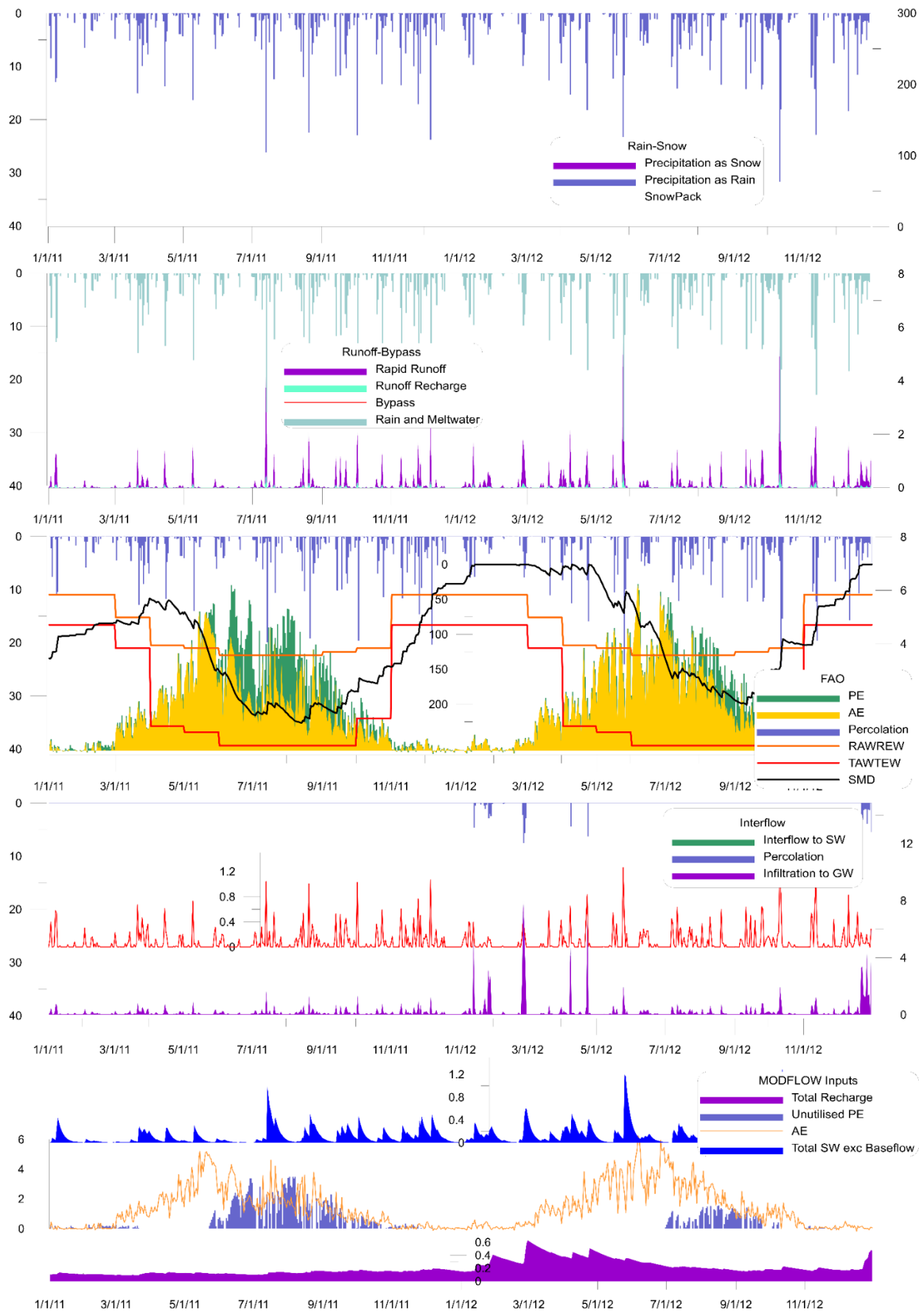


Figure 8.1 Timeseries breakdown of select flow processes within a catchment area

9 Code overview and Runtime

The **SWAcMod** library is written in Python 2.7. Python is arguably one of the most readable and easy-to-use programming languages. Its built-in, high-level data types (e.g. dictionaries) and dynamic typing (i.e. no need to declare the types of arguments and variables) allow Python code to be on average 3-5 times shorter than analogous Java code, and 5-10 times shorter than C++ code.

The downside is that its programs typically run slower than programs written in low-level implementation languages like C++ or Java, and this is particularly limiting in scientific and numerical computing. In order to mitigate this drawback, we have implemented the following 3 strategies: *parallelization*, *vectorization with NumPy* and *use of Cython*. Their combined effect is a 25-fold per core speedup of model time, defined as total time - time to read input files - time to write output files, bringing it down to a few milliseconds per node.

9.1 Node-wise parallelisation

Whenever possible, there is great computational advantage in splitting up a workload into multiple tasks, and distributing those tasks across multiple actors. This is known as parallel computing. There are two common ways of achieving this: threads and processes. While threads run in the same memory space, processes have separate memory. Due to the Global Interpreter Lock (GIL), only one thread can execute Python code at once and, in general, true multi-threading for CPU-bound applications is not possible. The solution to actually make use of the extra cores present in modern processors is to spawn multiple OS processes for each parallel task, side-stepping the GIL by feeding each process to a separate processor core, and then regrouping the results at the end once all processes are done. There are some drawbacks in using processes instead of threads, however. Each process takes longer to create, it introduces I/O overhead as data has to be communicated between processors, and it increases the memory footprint.

Since SWAcMod nodes are independent, the program is ‘embarrassingly parallelizable’, i.e. little effort is needed to split the problem into parallel tasks. We use the multiprocessing Python library to distribute SWAcMod nodes across cores. While the default value of processes is the number of cores of the machine where SWAcMod is running, the user can specify either the number of cores to use (with a positive number) or the ones to be left available (with a negative number). Nodes are shuffled across cores to maximize the probability of having all processes complete at the same time, and therefore not wasting computing power by leaving CPUs idle.

Since it would be prohibitive to have the whole output matrix in memory when running more than a few hundred nodes, and writing each node output to disk would be too slow, we only store aggregate quantities in memory. Communication between processes has to be ensured to sum these quantities, and this has been implemented with multiprocessing manager dictionaries.

9.2 NumPy vectorisation

Loops should be avoided as much as possible with interpreted languages like Python, since every iteration carries around extra baggage. Vectorisation is a style of programming where operations are applied to whole arrays at once, instead of writing loops on individual elements. Libraries like NumPy, heavily optimized for numerical computing, push loops down the stack to lower-level languages like C, and even further down to SIMD instructions at the CPU level whenever possible.

Vectorisation cannot be applied wholesale, a few quantities had to be calculated using a loop due to interdependency (see for example the function `get_interflow` in `model.pyx`, which calculates Interflow Store Volume, Infiltration Recharge and Interflow to Surface Water Courses at the same time).

9.3 Cython integration

Dynamic typing in Python makes the language extremely readable, but also fairly slow for scientific applications. Cython is a superset of the Python language that combines the best of both worlds. By supporting the declaration of C types on variables and class attributes, it allows its compiler to generate very efficient C code, while keeping the beautiful Python syntax.

For this reason, we chose to write the functions pertaining to the SWAc core model in Cython. All the Cython functions are grouped into the `model.pyx` file. This has proven to be particularly useful for all the quantities that could not be vectorised. We also used the special `cimport numpy` instruction to import compile-time information about the NumPy module into our Cython code.

10 Usage

```
swacmod_run.exe [-h] [-t] [-d] [-r] [-i INPUT_YML] [-o OUTPUT_DIR]
                [-f {hdf5,h5,csv}] [-s] [-v]
```

optional arguments:

<code>-h, --help</code>	show this help message and exit
<code>-t, --test</code>	run the whole model, but do not output any file
<code>-d, --debug</code>	verbose logging
<code>-r, --reduced</code>	output reduced format files
<code>-i, --input_yaml</code>	path to input yaml file inside input directory
<code>-o, --output_dir</code>	path to output directory
<code>-f, --format</code>	output file format, choose between "hdf5" (or "h5") and "csv"
<code>-s, --skip_prompt</code>	skip user prompts and warnings
<code>-v, --version</code>	display the current version

11 References

- Allen, R., Pereira, L., Raes, D. & Smith, M., 1998. *FAO Irrigation and Drainage Paper No. 56 - Crop Evapotranspiration (guidelines for computing crop water requirements)*. Rome: Food and Agriculture Organization of the United Nations.
- Beven, K. & Kirkby, M., 1979. A physically based, variable contributing area model of basin hydrology. *Hydrological Sciences Bulletin*, pp. 43-46.
- Bishop, T. & Rushton, K., 1993. Water resource study of the Nottinghamshire Sherwood Sandstone Aquifer System of Eastern England: Mathematical model of the Sherwood Sandstone Aquifer. *University of Birmingham, Department of Civil Engineering Report*.
- Harbaugh, A. & McDonald, M., 1996. *User's documentation for MODFLOW-96, an update to the U.S. Geological Survey modular finite-difference ground-water flow model*, Reston, VA: USGS.
- Heathcote, J., Lewis, R. & Soley, R., 2004. Rainfall routing to runoff and recharge for regional groundwater resource models. *Quarterly Journal of Engineering Geology and Hydrogeology*, 37(2), pp. 113-130.
- Hulme, P., Rushton, K. R. & Fletcher, S., 2001. *Estimating recharge in UK catchments..* Wallingford, IAHS, pp. 18-27.
- Langevin, C. et al., 2017. Documentation for the MODFLOW 6 Groundwater Flow Model. *Book 6, Modeling Techniques*, p. 197.
- Markstrom, S. et al., 2015. *PRMS-IV, the precipitation-runoff modeling system, version 4*, Reston, VA: USGS.
- Panday, S. et al., 2013. *MODFLOW-USG version 1: An unstructured grid version of MODFLaOW for simulating groundwater flow and tightly coupled processes using a control volume finite-difference formulation*, Reston, VA: USGS.
- Rushton, K., 2003. *Groundwater Hydrology: Conceptual and Computational Models*. Chichester, UK: John Wiley & Sons, Ltd.
- Walter, T. et al., 2005. Process-based snowmelt modeling: does it require more input data than temperature-index modeling?. *Journal of Hydrology*, 300(1-4), pp. 65-75.
- Wang, L. & Burke, S., 2017. A catchment-scale method to simulating the impact of historical nitrate loading from agricultural land on the nitrate-concentration trends in the sandstone aquifers in the Eden Valley, UK. *Science of The Total Environment*, Volume 579, pp. 133-148.
- WSP Environment & Infrastructure Solutions UK Limited, 2022. *4R - Routing of Rainfall to Runoff and Recharge User Guide*, Shrewsbury, UK: Environment Agency.
- Yang, C. et al., 2022. Nitrate transport velocity data in the global unsaturated zones. *Scientific Data*, 9(1).

Appendix A. Template master control file

```
#####
#                               YAML Configuration file for SWAcMod
#
# Variables marked as "This parameter can accept a path to [extension]"
# (see "rapid_runoff_params") can also contain a path to a file with that
# extension in their place.
#
# Example, for the "time_period" parameter, this can be:
#
# 1) a YAML object
#   time_period:
#     1: [1, 31]
#     2: [32, 60]
#     3: [61, 92]
#
# 2) A reference to a YAML file with the same object
#   time_period: time_period.yml
#
# 3) A reference to a CSV file with an analogous object
#   time_period: time_period.csv
#
#   where the CSV file contains (no header)
#   1,1,31
#   2,32,60
#   3,62,92
#
# Rules for converting YAML to CSV:
# - If the object is a dictionary (see "time_period"), keys will be the
#   first column in the CSV file.
#
# All paths can be either absolute or relative to this file. If the
# path is invalid, e.g. one of the directories does not exist, the program will
# not try to create it.
#
#####

#####
# Global parameters #
#####

# The version of the file format.
version: 2

# Pattern for output file names.
# This parameter is required.
run_name: my_run

# Directory to store temporary file-backed arrays
# This parameter is required.
temp_file_backed_array_directory: temp_scratch_files/

# Number of nodes in the 2D water accounting model
# This parameter is required.
num_nodes: 10

# Area of each node [L2]
# This parameter is required.
# This parameter can accept a path to YAML or CSV
# col #1: node number (integer, number of entries = num_nodes)
# col #2: node area (floating point) [L2]
node_areas: node_areas.yml

# X,Y of each node [L]
# This parameter is optional.
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: X coordinate (floating point) [L]
# col #3: Y coordinate (floating point) [L]
node_xy: node_xy.yml

# Start date for the time series. Has to be in format "YYYY-MM-DD".
# This parameter is required.
start_date: 1980-01-01

# Output data will be aggregated according to these time periods.
# This parameter is required.
# This parameter can accept a path to YAML or CSV
```

```

# col #1: first day of this time period (integer, number of entries = number of time periods)
[T]
# col #2: first day of next time period (integer, number of entries = number of time periods)
[T]
# (seconds, hours or other can be entered if used as standard unit of time in all files)
time_periods: time_periods.yml

# Number of cores to be used
# This parameter is optional.
# If not specified, all cores will be used
# If negative, all cores - abs(num_cores) will be used
num_cores: 1

# Output recharge file switch
# Can be either true or false
# This parameter is optional. If not specified, it will be set to "true".
output_recharge: true

# Output balance file for individual nodes. Accepts comma-separated values,
# ranges and overlapping values, e.g. "1, 3, 68-83, 5-7, 6".
# It also accepts "all" and "none" values.
# This parameter is optional. If not specified, it will be set to "none".
#output_individual: "1-10"

# IRCHCB (unit number in MODFLOW rch file for output to cell-by-cell file)
# This parameter is an integer.
# This parameter is optional. If not specified, it will be set to 50.
irchcb: 50

# Number of nodes per line of recharge file.
# This parameter is a positive integer.
# This parameter is optional. If not specified, it will be set to 10.
nodes_per_line: 5

# Map recharge to specific nodes.
# This parameter can accept a path to YAML or CSV
# Optional, if omitted assumes recharge goes to upper most active layer
# file is in the format SWAcNode,MFNode
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: MF node number (integer, number of entries = num_nodes)
recharge_node_mapping: rch_nodes.yml

# Output unit conversion factor
# This parameter is a positive float.
# This parameter is optional. If not specified, it will be set to 1.0
output_fac: 1.0

# Output data for all nodes on a single day. Has to be in format "YYYY-MM-DD".
# Also accepts "none" and "mean".
# This parameter is optional. If not specified, it will be set to "none".
spatial_output_date: 1980-01-01

#####
# Model parameters: Zones names and mappings #
#####

# Reporting zone mapping.
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# N.B. Nodes in zone 0 will not be calculated and will effectively be switched off
# Nodes not assigned to a zone, will by default be assigned to zone 0 and will not be calculated
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
reporting_zone_mapping: reporting_zone_mapping.yml

# Reporting zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in reporting_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
reporting_zone_mapping)
reporting_zone_names: reporting_zone_names.yml

# Rainfall zone mapping.
# This parameter is required
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)

```

```
# col #2: Zone number (integer, number of entries = num_nodes)
# col #3: Proportion of node area covered by zone (floating point, number of entries = num_nodes)
[-]
rainfall_zone_mapping: rainfall_zone_mapping.yml

# Rainfall zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in rainfall_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
rainfall_zone_mapping)
rainfall_zone_names: rainfall_zone_names.yml

# PE zone mapping.
# This parameter is required.
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
# col #3: Proportion of node area covered by zone (floating point, number of entries = num_nodes)
[-]
pe_zone_mapping: pe_zone_mapping.yml

# PE zone names.
# This parameter is optional.
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in pe_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
pe_zone_mapping)
pe_zone_names: pe_zone_names.yml

# Temperature zone mapping.
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
temperature_zone_mapping: temperature_zone_mapping.yml

# Temperature zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in temperature_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
temperature_zone_mapping)
temperature_zone_names: temperature_zone_names.yml

# Tmax_C zone mapping.
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
tmax_c_zone_mapping: tmax_c_zone_mapping.yml

# Tmax_C zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in tmax_c_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
tmax_c_zone_mapping)
tmax_c_zone_names: tmax_c_zone_names.yml

# Tmin_C zone mapping.
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
tmin_c_zone_mapping: tmin_c_zone_mapping.yml

# Tmin_C zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in tmin_c_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
tmin_c_zone_mapping)
tmin_c_zone_names: tmin_c_zone_names.yml

# windSp zone mapping.
# This parameter is optional
```

```

# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
windsp_zone_mapping: windsp_zone_mapping.yml

# windSp zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in windsp_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
windsp_zone_mapping)
windsp_zone_names: windsp_zone_names.yml

# Subroot zone mapping.
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
# col #3: Proportion of node area covered by zone (floating point, number of entries = num_nodes)
[-]
subroot_zone_mapping: subroot_zone_mapping.yml

# Subroot zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in subroot_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
subroot_zone_mapping)
subroot_zone_names: subroot_zone_names.yml

# Rapid Runoff Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
rapid_runoff_zone_mapping: rapid_runoff_zone_mapping.yml

# Rapid runoff zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in rapid_runoff_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
rapid_runoff_zone_mapping)
rapid_runoff_zone_names: rapid_runoff_zone_names.yml

# Runoff Recharge Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
swrecharge_zone_mapping: swrecharge_zone_mapping.yml

# Runoff recharge zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in swrecharge_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
swrecharge_zone_mapping)
swrecharge_zone_names: swrecharge_zone_names.yml

# Macropore Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
macropore_zone_mapping: macropore_zone_mapping.yml

# Macropore zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in macropore_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
macropore_zone_mapping)
macropore_zone_names: macropore_zone_names.yml

# Soil zone names.
# This parameter is optional

```

```
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries defines number of columns in soil_spatial -
# see FAO process)
# col #2: Zone name (text string, enclosed by ' ', number of entries defines number of columns
# in soil_spatial - see FAO process)
soil_zone_names: soil_zone_names.yml

# Land zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries defines number of columns in lu_spatial - see
# FAO process)
# col #2: Zone name (text string, enclosed by ' ', number of entries defines number of columns
# in lu_spatial - see FAO process)
landuse_zone_names: landuse_zone_names.yml

#####
# Process: Canopy interception and storage #
#####

# Canopy process: can be either "enabled" or "disabled"
# This parameter is required
canopy_process: enabled

# Canopy Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
canopy_zone_mapping: canopy_zone_mapping.yml

# Canopy zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in canopy_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
# canopy_zone_mapping)
canopy_zone_names: canopy_zone_names.yml

# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: Zone number (integer, number of entries = max zone number in canopy_zone_mapping)
# col #2: Free throughfall proportion (floating point, number of entries = max zone number in
# canopy_zone_mapping)
free_throughfall: free_throughfall.yml

# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: Zone number (integer, number of entries = max zone number in canopy_zone_mapping)
# col #2: Max canopy storage in mm (floating point, number of entries = max zone number in
# canopy_zone_mapping) [L]
max_canopy_storage: max_canopy_storage.yml

#####
# Process: Snow #
#####

# Simple snow process: can be either "enabled" or "disabled"
# This parameter is required
snow_process_simple: enabled

# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Starting snow pack in mm (integer, number of entries = num_nodes) [L]
# col #3: Max air temperature for snowfall in degrees C (integer, number of entries = num_nodes)
# [oC]
# col #4: Min air temperature for snowmelt in degrees C (integer, number of entries = num_nodes)
# [oC]
snow_params_simple: snow_params.yml

# Complex snow process: can be either "enabled" or "disabled"
# This parameter is required
snow_process_complex: disabled

# This parameter is optional
# This parameter can accept a path to YAML or CSV
```

```

# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Latitude in degrees (floating point, number of entries = num_nodes) [deg]
# col #3: Slope as vertical height over horizontal distance (floating point, number of entries
= num_nodes) [-]
# col #4: Aspect in degrees (inverted for S hemisphere) (floating point, number of entries =
num_nodes) [dd]
# col #5: Height of T measurement above ground in m (floating point, number of entries =
num_nodes) [L]
# col #6: Height of wind speed measurement above ground in m (floating point, number of entries
= num_nodes) [L]
# col #7: Ground albedo (floating point, number of entries = num_nodes) [-]
# col #8: Surface emissivity ratio (floating point, number of entries = num_nodes) [-]
# col #9: Proportion of tree cover (floating point, number of entries = num_nodes) [-]
# col #10: Starting snow depth in m (floating point, number of entries = num_nodes) [L]
# col #11: Starting snow density in kg/m3 (floating point, number of entries = num_nodes) [M/L3]
snow_params_complex: snow_params_complex.yml

#####
# Process: Proportion of rainfall converted to Runoff #
#####

# Rapid runoff process: can be either "enabled" or "disabled"
# This parameter is required
rapid_runoff_process: enabled

# Proportion of rainfall converted to runoff.
# This parameter is optional
# This parameter can accept a path to YAML
# Use example file for full input syntax
# One block per zone in rapid_runoff_zone_mapping
# For each block:
# Row #1: class_smd: followed by Soil Moisture Deficit class limits
# Row #2: class_ri: followed by rainfall intensity class limits
# Row #3 onwards: proportion of rainfall as runoff for each SMD class (1 row per rain intensity
class)
rapid_runoff_params: rapid_runoff_params.yml

#####
# Process: Runoff Recharge #
#####

# Runoff Recharge process: can be either "enabled" or "disabled"
# This parameter is required
swrecharge_process: enabled

# Runoff Recharge Proportion of RO
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in swrecharge_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: Proportion of runoff as recharge in this cell for zone 1 (floating point, number of
entries = 12) [-]
# Additional cols: Proportion of runoff as recharge in this cell for zones 2 to n, where n is
number of entries in swrecharge_zone_names (floating point, number of entries = 12) [-]
swrecharge_proportion: swrecharge_proportion.yml

# Runoff Recharge Limit [mm]
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in swrecharge_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: Maximum runoff recharge in mm for this cell in zone 1 (floating point, number of
entries = 12) [L]
# Additional cols: Maximum runoff recharge in mm for this cell in zones 2 to n, where n is
number of entries in swrecharge_zone_names (floating point, number of entries = 12) [L]
swrecharge_limit: swrecharge_limit.yml

#####
# Process: Single Cell Runoff Recharge #
#####

# Runoff Recharge in single cells process:
# Set to "disabled"
# Legacy feature (no longer maintained)
# This parameter is required
single_cell_swrecharge_process: disabled

```

```
#####
# Process: Macropore #
#####

# Macropore process: can be either "enabled" or "disabled"
# This parameter is required
macropore_process: enabled

# MacroPore: Proportion of Percolating Water Bypassing Root Zone
# and Interflow [mm]
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in macropore_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: Proportion of rainfall as macropore recharge in zone 1 (floating point, number of
entries = 12) [-]
# Additional cols: Proportion of rainfall as macropore recharge in zones 2 to n, where n is
number of entries in macropore_zone_names (floating point, number of entries = 12) [-]
macropore_proportion: macropore_proportion.yml

# MacroPore Bypass Limit [mm]
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in macropore_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: Maximum macropore recharge in zone 1 (floating point, number of entries = 12) [-]
# Additional cols: Maximum macropore recharge in zones 2 to n, where n is number of entries in
macropore_zone_names (floating point, number of entries = 12) [-]
macropore_limit: macropore_limit.yml

# MacroPore Bypass Activation [mm]
# This parameter is optional. If not specified, it will be set to 0.0.
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in macropore_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: SMD (or rainfall) threshold in mm for macropore recharge to occur in zone 1 (floating
point, number of entries = 12) [-]
# Additional cols: SMD (or rainfall) threshold in mm for macropore recharge to occur in zones
2 to n, where n is number of entries in macropore_zone_names (floating point, number of entries
= 12) [-]
macropore_activation: macropore_activation.yml

# MacroPore Proportion Direct Recharge
# This parameter is optional. If not specified, it will be set to 0.0.
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in macropore_zone_names
# col #1: Month number (integer, number of entries = 12)
# col #2: Proportion of rainfall assigned to macropore recharge in zone 1 (floating point,
number of entries = 12) [-]
# Additional cols: Proportion of rainfall assigned to macropore recharge in zones 2 to n, where
n is number of entries in macropore_zone_names (floating point, number of entries = 12) [-]
macropore_recharge: macropore_recharge.yml

# MacroPore Activation Option
# This parameter is optional. If not specified, it will be set to SMD.
# RI = rainfall intensity, SMD = previous day's SMD
#macropore_activation_option: "RI"

#####
# Process: FAO: SMD-AE Accounting #
#####

# FAO process: can be either "enabled" or "disabled"
# This parameter is required
fao_process: enabled

# Input landuse and soil with "ls" or just landuse with "l"
# This parameter is optional. If not specified, it defaults to "ls".
fao_input: 'l'

# Soil Properties (FC, WP, p)
# This parameter is optional.
# If not specified and "fao_input" is "ls", "fao_process" will be set to
# "disabled"
# This parameter can accept a path to YAML or CSV
# row #1: 'WP' followed by wilting point in m3/m3 for each soil zone (floating point, 1 column
for each entry in soil_zone_names) [L3/L3]
```

```
# row #2: 'p' followed by RAW as a fraction TAW for each soil zone (floating point, 1 column
for each entry in soil_zone_names) [-]
# row #3: 'FC' followed by field capacity in m3/m3 for each soil zone (floating point, 1 column
for each entry in soil_zone_names) [L3/L3]
soil_static_params: soil_static_params.yml

# SMD Properties (starting SMD)
# This parameter is optional.
# If not specified, it will be set to 0.0
# If not specified and "fao_input" is "ls", "fao_process" will be set to
# "disabled"
# This parameter can accept a path to YAML or CSV
# row #1: 'starting_SMD' followed by initial SMD in mm for each soil zone (integer, 1 column
for each entry in soil_zone_names) [L]
smd: smd.yml

# Soil Spatial
# This parameter is optional.
# If not specified and "fao_input" is "ls", "fao_process" will be set to
# "disabled"
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# Additional cols: Proportion of this node in each soil zone (floating point, 1 column for each
entry in soil_zone_names)
soil_spatial: soil_spatial.yml

# LU Spatial
# This parameter is optional.
# If not specified, "fao_process" will be set to "disabled"
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# Additional cols: Proportion of this node in each soil zone (floating point, 1 column for each
entry in landuse_zone_names) [-]
lu_spatial: lu_spatial.yml

# LU Time Variant: Zr
# This parameter is optional.
# If not specified and "fao_input" is "ls", "fao_process" will be set to
# "disabled"
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in landuse_zone_names
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Rooting depth in m for each land use zone (floating point, 1 column for each
entry in landuse_zone_names) [L]
zr: zr.yml

# LU Time Variant: Kc
# This parameter is optional.
# If not specified, "fao_process" will be set to "disabled"
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in landuse_zone_names
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Crop coefficient for each land use zone (floating point, 1 column for each
entry in landuse_zone_names) [-]
kc: kc.yml

# TAW
# This parameter is optional.
# If not specified and "fao_input" is "l", "fao_input" will be set to "ls".
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in landuse_zone_names
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Total Available soil Water in mm for each land use zone (floating point, 1
column for each entry in landuse_zone_names) [-]
taw: taw.yml

# RAW
# This parameter is optional.
# If not specified and "fao_input" is "l", "fao_input" will be set to "ls".
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in landuse_zone_names
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Readily Available soil Water in mm for each land use zone (floating point,
1 column for each entry in landuse_zone_names) [-]
raw: raw.yml

# Percolation Limit (Rejected Recharge initiation)
```

```
# This parameter is optional.
# This parameter can accept a path to YAML or CSV
# Single column: percolation_rejection keyword, followed by percolation limit in mm for each
land use zone (floating point, 1 column for each entry in landuse_zone_names) [L]
percolation_rejection: percolation_rejection.yml

# Assign percolation rejection as time series
# Can be either true or false
percolation_rejection_use_timeseries: true

# row #1: percolation_rejection_ts: keyword
# 1 additional row for each day (or user-defined time unit) in time_periods, followed by
percolation limit in mm for each land use zone (floating point, 1 column for each entry in
landuse_zone_names) [L]
percolation_rejection_ts: percolation_rejection_ts.yml

#####
# Process: Subroot Leakage #
#####

# Leakage process: can be either "enabled" or "disabled"
# This parameter is required
subroot_leakage_process: enabled

# Fraction of leakage to Permit
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Proportion of subroot leakage permitted in each cell, linked to maximum rate listed
in subroot_leakage_ts (floating point, number of entries = num_nodes) [-]
subroot_leakage_fraction: subroot_leakage_fraction.yml

#####
# Process: Interflow #
#####

# Interflow process: can be either "enabled" or "disabled"
# This parameter is required
interflow_process: enabled

# Interflow parameters (Initial interflow store, Store bypass, Infiltration,
# Interflow to rivers )

# Interflow Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
interflow_zone_mapping: interflow_zone_mapping.yml

# Runoff recharge zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in interflow_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
interflow_zone_mapping)
interflow_zone_names: interflow_zone_names.yml

# col #1: Zone number (integer, number of entries = max zone number in interflow_zone_mapping)
# col #2: Initial value for interflow store in mm (floating point, number of entries = max zone
number in interflow_zone_mapping) [L]
init_interflow_store: init_interflow_store.yml

# col #1: Zone number (integer, number of entries = max zone number in interflow_zone_mapping)
# col #2: Proportion of water from root zone percolation and subrootzone leakage bypassing the
interflow store (floating point, number of entries = max zone number in interflow_zone_mapping)
[L]
interflow_store_bypass: interflow_store_bypass.yml

# col #1: Zone number (integer, number of entries = max zone number in interflow_zone_mapping)
# col #2: Maximum infiltration to groundwater from interflow store in mm (floating point, number
of entries = max zone number in interflow_zone_mapping) [L]
infiltration_limit: infiltration_limit.yml

# col #1: Zone number (integer, number of entries = max zone number in interflow_zone_mapping)
```

```

# col #2: Proportion of interflow store in each day (or user-defined time unit) released to
surface water (floating point, number of entries = max zone number in interflow_zone_mapping)
[-]
interflow_decay: interflow_decay.yml

# Assign infiltration limit as time series
# Can be either true or false
infiltration_limit_use_timeseries: true

# Assign interflow decay as time series
# Can be either true or false
interflow_decay_use_timeseries: true

# row #1: infiltration_limit_ts: keyword
# 1 additional row for each day (or user-defined time unit) in time_periods, followed by
infiltration limit in mm for each interflow zone (floating point, 1 column for each entry in
interflow_zone_names) [L]
infiltration_limit_ts: infiltration_limit_ts.yml

# row #1: interflow_decay_ts: keyword
# 1 additional row for each day (or user-defined time unit) in time_periods, followed by
interflow decay proportion for each interflow zone (floating point, 1 column for each entry in
interflow_zone_names) [-]
interflow_decay_ts: interflow_decay_ts.yml

#####
# Process: Recharge Release to Saturated Zone #
#####

# Interflow process: can be either "enabled" or "disabled"
# This parameter is required
recharge_attenuation_process: enabled

# Recharge Parameters
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Initial recharge store in mm (floating point, number of entries = num_nodes) [L]
# col #3: Proportion of recharge store released each day (or user-defined time unit) (floating
point, number of entries = num_nodes) [-]
# col #4: Maximum rate of release in mm (floating point, number of entries = num_nodes) [L]
recharge_attenuation_params: recharge_attenuation_params.yml

#####
# Process: Solute #
#####

# historical solute process: can be either "enabled" or "disabled"
# This parameter is required
# If enabled, solute process must also be enabled and its outputs will be made available to a
sequential (e.g. future-facing) run
historical_solute_process: disabled

# Solute process: can be either "enabled" or "disabled"
# This parameter is required
# If enabled, entries required for all subsequent parameters
solute_process: disabled

# Floating-point value, representing logarithm base in cumulative distribution function of
unsaturated zone K, where  $F(K) = 0.5 * [1 + \text{erf}(-(\log(K) - \mu) / (\sigma * \text{SQRT}(2)))]$ 
solute_calibration_a: 1.38

# Column 1: SWAc node number (integer, number of entries = num_nodes)
# Column 2: fitting parameter mu in above equation (floating point, number of entries =
num_nodes) [L]
solute_calibration_mu: solute_mu.yml

# Floating-point value, fitting parameter sigma in above equation
solute_calibration_sigma: 3.96

# Floating-point value, fitting parameter alpha in above equation
solute_calibration_alpha: 3906.25

# Column 1: SWAc node number (integer, number of entries = num_nodes)
# Column 2: effective porosity
solute_calibration_effective_porosity: solute_effective_porosity.yml

```

```
# Column 1: SWAc node number (integer, number of entries = num_nodes)
# Column 2: depth to water in m (floating point, number of entries = num_nodes) [L]
solute_depth_to_water: solute_average_DTW.yml

# Column 1: SWAc node number (integer, number of entries = num_nodes)
# Column 2: UNIQUE (not used in SWAcMod)
# Column 3: X (not used in SWAcMod)
# Column 4: Y (not used in SWAcMod)
# Column 5: Max load (kg/hectare)
# Column 6: HER at 5% (mm/day)
# Column 7: HER at 50% (mm/day)
# Column 8: HER at 95% (mm/day)
# Column 9: 5PercLoadM (not used in SWAcMod)
# Column 10: 50PercLoad (not used in SWAcMod)
# Column 11: 95PercLoad (not used in SWAcMod)
solute_loading: solute_loading.yml

#####
# Process: SW Release #
#####

# SW release process: can be either "enabled" or "disabled"
# This parameter is required
sw_process: enabled

# SW Parameters
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Column 1: SWAc node number (integer, number of entries = num_nodes)
# Column 2: Initial SW store in mm (floating point, number of entries = num_nodes) [L]
# Column 3: Proportion of SW store released each day (or user-defined time unit) (floating
point, number of entries = num_nodes) [-]
sw_params: sw_params.yml

#####
# Process: SW Ponding #
#####

# SW release process: can be either "enabled" or "disabled"
# This parameter is required
# If enabled all FAO inputs should reflect open water
sw_ponding_process: disabled

# SW Release Zone Mapping
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = num_nodes)
# col #2: Zone number (integer, number of entries = num_nodes)
sw_zone_mapping: sw_zone_mapping.yml

# SW release zone names.
# This parameter is optional
# This parameter can accept a path to YAML
# col #1: Zone number (integer, number of entries = max zone number in sw_zone_mapping)
# col #2: Zone name (text string, enclosed by ' ', number of entries = max zone number in
sw_zone_mapping)
sw_zone_names: sw_zone_names.yml

# SW Release downstream proportion
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in sw_zone_names
# row #1: sw_downstream: keyword
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Proportion of ponded water released in each time unit (floating point, 1
column for each entry in sw_zone_names) [-]
sw_downstream: sw_downstream.yml

# SW Release Activation threshold [mm]
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# Number of cols defined by number of entries in sw_zone_names
# row #1: sw_activation: keyword
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Minimum water depth in mm before bed infiltration can occur (floating point,
1 column for each entry in sw_zone_names) [L]
sw_activation: sw_activation.yml
```

```
# SW Release Proportion Bed Infiltration
# Number of cols defined by number of entries in sw_zone_names
# row #1: sw_bed_infiltration: keyword
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Proportion of ponded water infiltrating through bed to recharge attenuation
store (floating point, 1 column for each entry in sw_zone_names) [-]
sw_bed_infiltration: sw_bed_infiltration.yml

# SW Release Proportion Direct Recharge
# Number of cols defined by number of entries in sw_zone_names
# row #1: sw_direct_recharge: keyword
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Proportion of ponded water leaving bed and bypassing recharge attenuation
store (floating point, 1 column for each entry in sw_zone_names) [-]
sw_direct_recharge: sw_direct_recharge.yml

# SW Release PE to open water
# Number of cols defined by number of entries in sw_zone_names
# row #1: sw_pe_to_open_water: keyword
# col #1: Month number (integer, number of entries = 12)
# Additional cols: Open water evaporation coefficient for each zone (floating point, 1 column
for each entry in sw_zone_names) [-]
sw_pe_to_open_water: sw_pe_to_open_water.yml

# # SW Release Initial ponding depth [mm]
sw_init_ponding: 5.0

# # SW Release Max ponding depth [mm]
sw_max_ponding: 300.0

# # SW Release Area subject to ponding [fraction]
# row #1: sw_ponding_area:: keyword
# col #1: Zone number (integer, number of entries = max zone number in sw_zone_mapping)
# col #2: Proportion of cell subject to ponding (floating point, 1 column for each entry in
sw_zone_names) [-]
sw_ponding_area: sw_ponding_area.yml

#####
# Process: Surface Water Routing #
#####

# Routing process: can be either "enabled" or "disabled"
# This parameter is required
routing_process: enabled

# Routing network topology and SFR parameters
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = number of nodes subject to EVT)
# col #2: downstr - swac node downstream of this one
# col #3: str_flag - 1 = stream cell, 0 = *not* stream cell
# col #4: node_mf - modflow node number of this node
# col #5: RCHLEN - the modflow SFR variable RCHLEN
# col #6: ca - the upstream contributing area
# col #7: STRTOP - the modflow SFR variable STRTOP
# col #8: STRTHICK - the modflow SFR variable STRTHICK
# col #9: STRHCL1 - the modflow SFR variable STRHCL1
# col #10: DEPTH2 - the modflow SFR variable DEPTH2
# col #11: WIDTH2 - the modflow SFR variable WIDTH2
routing_topology: routing_parameters.yml

# Output modflow sfr2 file switch
# Can be either true or false
# This parameter is optional. If not specified, it will be set to "false".
output_sfr: true

# Attenuate sfr flows
# Can be either true or false
# This parameter only applies when:
#   - output_sfr is true
# This parameter is optional.
attenuate_sfr_flows: false

# Zones for each cell in the model.
# Column 1 is the node number (not the stream cell number)
# Column 2 is the zone number
```

```
# Rows only need to be supplied for nodes that are part of a stream. There is no need to cover
the whole model.
# This parameter is optional.
# sfr_flow_zones: sfr_flow_zones.yml

# Release proportions for each zone and month.
# Column 1 is the month number (January is 1)
# The remaining columns are the release proportion, one for each zone.
# sfr_flow_monthly_proportions: sfr_flow_monthly_proportions.yml

# Modflow 6 sfr obs input file
# This parameter is optional.
sfr_obs: gauges_sfr.obs

# istcb1 (MODFLOW unit number)
# Has to be an integer.
# This parameter is optional. If not specified, it will be set to 50.
istcb1: 50

# istcb2 (MODFLOW unit number)
# Has to be an integer.
# This parameter is optional. If not specified, it will be set to 55.
istcb2: 55

# Surface water discharges time-series m3/day
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# One column for each SW discharge location in swdis_locs
# SW discharge rate in m3/day (or other user-defined time unit) (floating point, number of rows
= maximum value in time_periods)
swdis_ts: swdis_ts.yml

# Surface water discharges points
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = number of nodes with SW discharges)
# col #2: Index number of discharge from 1 to number of nodes with SW discharges (integer,
number of entries = number of nodes with SW discharges)
swdis_locs: swdis_locs.yml

# swdis_f
# Surface water discharges time-series frequency
# Has to be an integer: 0 = daily, 1 = weekly, 2 = monthly
# This parameter is optional. If not specified, it will be set to 0 (daily).
#swdis_f: 0

# Surface water abstractions time-series m3/day
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# One column for each SW abstraction location in swabs_locs
# SW abstraction rate in m3/day (or other user-defined time unit) (floating point, number of
rows = maximum value in time_periods)
swabs_ts: swabs_ts.yml

# swabs_f
# Surface water abstractions time-series frequency
# Has to be an integer: 0 = daily, 1 = weekly, 2 = monthly
# This parameter is optional. If not specified, it will be set to 0 (daily).
#swabs_f: 0

# Surface water discharges points
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = number of nodes with SW abstractions)
# col #2: Index number of abstraction from 1 to number of nodes with SW abstractions (integer,
number of entries = number of nodes with SW abstractions)
swabs_locs: swabs_locs.yml

#####
# Time series #
#####

# This parameter is required.
# This parameter can accept a path to YAML or CSV
# One column for each entry in rainfall_zone_names
# Rainfall in mm (floating point, number of rows = maximum value in time_periods) [L]
rainfall_ts: rainfall_ts.yml

# This parameter is optional.
```

```
# This parameter is required only if the FAO or the Canopy process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in pe_zone_names
# Rainfall in mm (floating point, number of rows = maximum value in time_periods) [L]
pe_ts: pe_ts.yml

# This parameter is optional.
# This parameter is required only if the Snow process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in temperature_zone_names
# Temperature in C (floating point, number of rows = maximum value in time_periods) [C]
temperature_ts: temperature_ts.yml

# This parameter is optional.
# This parameter is required only if the Snow process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in tmax_c_zone_names
# Max temperature in C (floating point, number of rows = maximum value in time_periods) [C]
tmax_c_ts: tmax_c_ts.yml

# This parameter is optional.
# This parameter is required only if the Snow process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in tmin_c_zone_names
# Min temperature in C (floating point, number of rows = maximum value in time_periods) [C]
tmin_c_ts: tmin_c_ts.yml

# This parameter is optional.
# This parameter is required only if the Snow process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in windsp_zone_names
# Wind speed in m/s (floating point, number of rows = maximum value in time_periods) [L/T]
windsp_ts: windsp_ts.yml

# This parameter is optional.
# This parameter is required only if the Leakage process is enabled.
# This parameter can accept a path to YAML or CSV
# One column for each entry in subroot_zone_names
# Maximum subroot leakage rate in mm (floating point, number of rows = maximum value in
time_periods) [L]
subroot_leakage_ts: subroot_leakage_ts.yml

# Single keyword: mf6, mf96 or mfusg
gwmodel_type: mf6

# Only read if mf96 entered above.
# Number of layers, rows and columns in model
mf96_lrc: [2, 2, 5]

#####
#EVT      #
#####

# Output evt file switch
# Can be either true or false
# This parameter is optional. If not specified, it will be set to "false".
output_evt: true

# allow excess sw to modflow evt process
excess_sw_process: disabled

# evt parameters (ievt, surf, exdp)
# This parameter is optional
# This parameter can accept a path to YAML or CSV
# col #1: SWAc node number (integer, number of entries = number of nodes subject to EVT)
# col #2: MF node number (integer, number of entries = number of nodes subject to EVT)
# col #3: Elevation of the ET surface in m (floating point, number of entries = number of nodes
subject to EVT) [L]
# col #4: ET extinction depth in m (floating point, number of entries = number of nodes subject
to EVT) [L]
evt_parameters: evt_params.yml

# ievtcb (unit number in MODFLOW evt file for output to cell-by-cell file)
# This parameter is an integer.
# This parameter is optional. If not specified, it will be set to 50.
#ievtcb: 50
```

```
# nevtopt (MODFLOW ET option code)
# This parameter is an integer.
# This parameter is optional. If not specified, it will be set to 2.
#nevtopt: 2
```

Appendix B. Description of Input and Output

Block: Global parameters

These user inputs govern the general behaviour of the model run and include the model name, timing, model mesh geometry, and assignment of computer processing resources. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

version is an *optional* input, provided as an integer. The value provided defines the version of input file being used. If not provided, version 1 is assumed.

run_name is a *required* input, provided as a text string. This text string will appear in all output files produced by this run.

temp_file_backed_array_directory is a *required* input, provided as a text string. This text string comprises a file path for SWAc to store temporary file-backed arrays.

num_nodes is a *required* input, provided as an integer. The value provided defines the number of nodes in the 2D water accounting model.

node_areas is a *required* input, provided as a path to either a YAML or CSV file. It defines the area of each node and is used by SWAc to convert 1D fluxes to volumetric rates.

node_xy is an *optional* input, provided as a path to either a YAML or CSV file. It specifies the spatial coordinates of each node, although this information is not used by SWAc in its calculations. The file should comprise 3 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *X coordinate (floating point) [L]*

Column 3: *Y coordinate (floating point) [L]*

start_date is a *required* input, provided in the format: "YYYY-MM-DD". It specifies the starting date of the simulation and is used by SWAc to specify the dates of all time periods when generating output files.

time_periods is a *required* input, provided as a path to either a YAML or CSV file. It defines the periods over which output data will be aggregated (analogous to stress periods in MODFLOW). The file should comprise 2 columns, as follows:

Column 1: *first day of this time period (integer, number of entries = number of time periods) [T]*

Column 2: *first day of next time period (i.e. end of current time period plus 1, notably in the final time period of the model) (integer, number of entries = number of time periods) [T]*

The normal unit of time is days, although seconds, hours or other can be entered if used as the standard unit of time in all files.

num_cores is an *optional* input, provided as an integer. It defines the number of cores to be used. If unspecified: All cores will be used. If negative: The absolute value of (num_cores) will be used.

output_recharge is an *optional* input, provided as a boolean ("true" or "false"). This switch determines whether the model should produce a recharge file as part of its output. If not specified, it will be set to "true".

output_individual is an *optional* input, provided as a string. It accepts comma-separated values, ranges and overlapping values, e.g. "1, 3, 68-83, 5-7, 6". It also accepts "all" and "none" values. If not specified, it will be set to "none".

IRCHCB is an *optional* input, provided as an integer. It is used to control the output of cell-by-cell flow terms. If not specified: it will be set to 50.

nodes_per_line is an *optional* input, given as a positive integer. It defines the number of nodes per line of the recharge file. If not specified: it will be set to 10.

recharge_node_mapping is an *optional* input, provided as a path to a YAML or CSV file. This parameter specifies a file that maps recharge to specific nodes in the model. Type: Path to a YAML or CSV file
Optional: If not specified: recharge is assumed to go to the uppermost active layer

The file should comprise of two columns:

Column 1: SWAc node number (integer, number of entries = num_nodes)

Column 2: MF node number (integer, number of entries = num_nodes)

output_fac is an *optional* input, provided as a positive float, as an output unit conversion factor. If not specified: value will be set to 1.0

spatial_output_date is an *optional* input, provided in the format "YYYY-MM-DD". Will also accept "none" or "mean". When a valid date is provided, the system will return spatial output data for all nodes in the model or system for the specified date only. If not specified: value will be set to "none".

Block: *Model parameters: Zones names and mappings*

These user inputs define the spatial zone definition for the various processes. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

reporting_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to define how nodes in the model are grouped into reporting zones. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (integer, number of entries = num_nodes)

Column 2: zone number (integer, number of entries = num_nodes)

Nodes assigned to zone 0 will not be calculated and effectively be switched off. Nodes not assigned to a reporting zone, will by default be assigned to zone 0.

reporting_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to the mapped zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in reporting_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in reporting_zone_mapping)*

rainfall_zone_mapping is a *required* input, provided as a path to either a YAML or CSV file. The file should comprise 3 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

Column 3: *Proportion of node area covered by zone (floating point, number of entries = num_nodes)*

rainfall_zone_names is an *optional* input, provided as a path to YAML or a CSV file. Allows user to define names for rainfall zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in rainfall_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in rainfall_zone_mapping)*

pe_zone_mapping is a *required* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

Column 3: *Proportion of node area covered by zone (floating point, number of entries = num_nodes)*

pe_zone_names is an *optional* input provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in pe_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in pe_zone_mapping)*

temperature_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign temperature zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

temperature_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to temperature zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in temperature_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in temperature_zone_mapping)*

tmax_c_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign maximum temperature zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (*integer, number of entries = num_nodes*)

Column 2: Zone number (*integer, number of entries = num_nodes*)

tmax_c_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to maximum temperature zones. The file should comprise 2 columns, as follows:

Column 1: Zone number (*integer, number of entries = max zone number in tmax_c_zone_mapping*)

Column 2: Zone name (*text string, enclosed by ' ', number of entries = max zone number in tmax_c_zone_mapping*)

tmin_c_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign minimum temperature zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (*integer, number of entries = num_nodes*)

Column 2: Zone number (*integer, number of entries = num_nodes*)

tmin_c_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to minimum temperature zones. The file should comprise 2 columns, as follows:

Column 1: Zone number (*integer, number of entries = max zone number in tmin_c_zone_mapping*)

Column 2: Zone name (*text string, enclosed by ' ', number of entries = max zone number in tmin_c_zone_mapping*)

windsp_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign wind speed zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (*integer, number of entries = num_nodes*)

Column 2: Zone number (*integer, number of entries = num_nodes*)

windsp_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to wind speed zones. The file should comprise 2 columns, as follows:

Column 1: Zone number (*integer, number of entries = max zone number in windsp_zone_mapping*)

Column 2: Zone name (*text string, enclosed by ' ', number of entries = max zone number in windsp_zone_mapping*)

subroot_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign subroot zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (*integer, number of entries = num_nodes*)

Column 2: Zone number (*integer, number of entries = num_nodes*)

subroot_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to subroot zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in subroot_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in subroot_zone_mapping)*

rapid_runoff_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign rapid runoff zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

rapid_runoff_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to rapid runoff zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in rapid_runoff_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in rapid_runoff_zone_mapping)*

swrecharge_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign runoff recharge zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

swrecharge_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to runoff recharge zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in swrecharge_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in swrecharge_zone_mapping)*

macropore_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign macropore zones to SWAc nodes. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

macropore_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to macropore zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in macropore_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in macropore_zone_mapping)*

soil_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to soil zones. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries defines number of columns in soil_spatial - see
FAO process)*

Column 2: Zone name (text string, enclosed by ' ', number of entries defines number of columns in *soil_spatial* - see *FAO process*)

landuse_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to landuse zones. The file should comprise 2 columns, as follows:

Column 1: Zone number (integer, number of entries defines number of columns in *lu_spatial* - see *FAO process*)

Column 2: Zone name (text string, enclosed by ' ', number of entries defines number of columns in *lu_spatial* - see *FAO process*)

Block: *Process*: Canopy interception and storage

These user inputs define parameters that govern the canopy interception process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

canopy_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the canopy process is active in the model.

canopy_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign canopy zones to nodes. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (integer, number of entries = *num_nodes*)

Column 2: Zone number (integer, number of entries = *num_nodes*)

canopy_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to assign names to canopy zones. The file should comprise 2 columns, as follows:

Column 1: Zone number (integer, number of entries = *max zone number in canopy_zone_mapping*)

Column 2: Zone name (text string, enclosed by ' ', number of entries = *max zone number in canopy_zone_mapping*)

free_throughfall is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the free throughfall proportion for each canopy zone. The file should comprise 2 columns, as follows:

Column 1: Zone number (integer, number of entries = *max zone number in canopy_zone_mapping*)

Column 2: Free throughfall proportion (floating point, number of entries = *max zone number in canopy_zone_mapping*)

max_canopy_storage is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the maximum canopy storage for each canopy zone. The file should comprise 2 columns, as follows:

Column 1: Zone number (integer, number of entries = *max zone number in canopy_zone_mapping*)

Column 2: Max canopy storage in mm (floating point, number of entries = *max zone number in canopy_zone_mapping*) [L]

Block: *Process*: Snow

These user inputs define parameters that govern the simple and complex snow melt processes. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

snow_process_simple is a *required* input. It can be set to either "enabled" or "disabled" to control whether the simple snow process is active in the model.

snow_params_simple is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify parameters for the simple snow process. The file should comprise 4 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Starting snow pack in mm (integer, number of entries = num_nodes) [L]*

Column 3: *Max air temperature for snowfall in degrees C (integer, number of entries = num_nodes) [oC]*

Column 4: *Min air temperature for snowmelt in degrees C (integer, number of entries = num_nodes) [oC]*

snow_process_complex is a *required* input. It can be set to either "enabled" or "disabled" to control whether the complex snow process is active in the model.

snow_params_complex is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify parameters for the complex snow process. The file should comprise 11 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Latitude in degrees (floating point, number of entries = num_nodes) [deg]*

Column 3: *Slope as vertical height over horizontal distance (floating point, number of entries = num_nodes) [-]*

Column 4: *Aspect in degrees (inverted for S hemisphere) (floating point, number of entries = num_nodes) [dd]*

Column 5: *Height of T measurement above ground in m (floating point, number of entries = num_nodes) [L]*

Column 6: *Height of wind speed measurement above ground in m (floating point, number of entries = num_nodes) [L]*

Column 7: *Ground albedo (floating point, number of entries = num_nodes) [-]*

Column 8: *Surface emissivity ratio (floating point, number of entries = num_nodes) [-]*

Column 9: *Proportion of tree cover (floating point, number of entries = num_nodes) [-]*

Column 10: *Starting snow depth in m (floating point, number of entries = num_nodes) [L]*

Column 11: *Starting snow density in kg/m³ (floating point, number of entries = num_nodes) [M/L³]*

Block: *Process*: Proportion of rainfall converted to Runoff

These user inputs define parameters that govern the rapid runoff process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

rapid_runoff_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the rapid runoff process is active in the model.

rapid_runoff_params is an *optional* input, provided as a path to a YAML file. It allows the user to specify parameters for the rapid runoff process. The file should be structured as follows:

One block per zone in *rapid_runoff_zone_mapping*:

For each block:

Row 1: *class_smd*: followed by Soil Moisture Deficit class limits

Row 2: *class_ri*: followed by rainfall intensity class limits

Row 3 onwards: *proportion of rainfall as runoff for each SMD class* (1 row per rain intensity class)

Block: *Process*: Runoff recharge

These user inputs define parameters that govern the runoff recharge process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

swrecharge_process is a required input. It can be set to either "enabled" or "disabled" to control whether the runoff recharge process is active in the model.

swrecharge_proportion is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the proportion of runoff as recharge for each zone. The file should comprise multiple columns, as follows:

Column 1: *Month number* (integer, number of entries = 12)

Column 2: *Proportion of runoff as recharge in this cell for zone 1* (floating point, number of entries = 12) [-]

Additional columns: *Proportion of runoff as recharge in this cell for zones 2 to n*, where *n* is number of entries in *swrecharge_zone_names* (floating point, number of entries = 12) [-]

swrecharge_limit is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the maximum runoff recharge for each zone. The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Column 2: *Maximum runoff recharge in mm for this cell in zone 1 (floating point, number of entries = 12) [L]*

Additional columns: *Maximum runoff recharge in mm for this cell in zones 2 to n, where n is number of entries in swrecharge_zone_names (floating point, number of entries = 12) [L]*

Block: *Process*: Single Cell Runoff Recharge

This block refers to the single cell runoff recharge process. This is a legacy feature that is no longer maintained

Explanation of keywords

single_cell_swrecharge_process is a *required* input. It should be set to "disabled" as it is a legacy feature that is no longer maintained.

Block: *Process*: Macropore

These user inputs define parameters that govern the macropore recharge process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

macropore_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the macropore process is active in the model.

macropore_proportion is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the proportion of percolating water that bypasses the Root Zone and Interflow. The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Column 2: *Proportion of rainfall as macropore recharge in zone 1 (floating point, number of entries = 12) [-]*

Additional columns: *Proportion of rainfall as macropore recharge in zones 2 to n, where n is number of entries in macropore_zone_names (floating point, number of entries = 12) [-]*

macropore_limit is an *optional* input, provided as a path to either a YAML or CSV file. It allows the user to specify the maximum macropore bypass limit for each macropore zone. The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Column 2: *Maximum macropore recharge in zone 1 (floating point, number of entries = 12) [-]*

Additional columns: *Maximum macropore recharge in zones 2 to n, where n is number of entries in macropore_zone_names (floating point, number of entries = 12) [-]*

macropore_activation is an *optional* input, provided as a path to either a YAML or CSV file. If not specified, it will be set to 0.0. It allows the user to specify the SMD (or rainfall intensity) threshold for

macropore recharge activation for each macropore zone. The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Column 2: *SMD (or rainfall) threshold in mm for macropore recharge to occur in zone 1 (floating point, number of entries = 12) [-]*

Additional columns: *SMD (or rainfall) threshold in mm for macropore recharge to occur in zones 2 to n, where n is number of entries in macropore_zone_names (floating point, number of entries = 12) [-]*

macropore_recharge is an *optional* input, provided as a path to either a YAML or CSV file. If not specified, it will be set to 0.0. It allows the user to specify the proportion of rainfall assigned to macropore recharge for each macropore zone. The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Column 2: *Proportion of rainfall assigned to macropore recharge in zone 1 (floating point, number of entries = 12) [-]*

Additional columns: *Proportion of rainfall assigned to macropore recharge in zones 2 to n, where n is number of entries in macropore_zone_names (floating point, number of entries = 12) [-]*

macropore_activation_option is an *optional* input. If not specified, it will be set to SMD. It can be set to either "RI" (rainfall intensity) or "SMD" (previous day's SMD) to determine the activation method for macropore processes.

Block: *Process*: FAO: SMD-AE Accounting

These user inputs define parameters that govern the FAO process, as well as the percolation rejection process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

fao_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the FAO process is active in the model.

fao_input is an *optional* input. If not specified, it defaults to "Is". It can be set to either "Is" (input land use and soil) or "I" (input just landuse).

soil_static_params is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "Is", "fao_process" will be set to "disabled". The file should comprise 3 rows, as follows:

Row 1: *'WP' followed by wilting point in m3/m3 for each soil zone (floating point, 1 column for each entry in soil_zone_names) [L3/L3]*

Row 2: *'p' followed by RAW as a fraction TAW for each soil zone (floating point, 1 column for each entry in soil_zone_names) [-]*

Row 3: *'FC' followed by field capacity in m3/m3 for each soil zone (floating point, 1 column for each entry in soil_zone_names) [L3/L3]*

smd is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "ls", "fao_process" will be set to "disabled". The file should comprise 1 row, as follows:

Row 1: *'starting_SMD' followed by initial SMD in mm for each soil zone (integer, 1 column for each entry in soil_zone_names) [L]*

soil_spatial is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "ls", "fao_process" will be set to "disabled". The file should comprise multiple columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Additional columns: *Proportion of this node in each soil zone (floating point, 1 column for each entry in soil_zone_names)*

lu_spatial is an *optional* input, provided as a path to either a YAML or CSV file. If not specified: "fao_process" will be set to "disabled". The file should comprise multiple columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Additional columns: *Proportion of this node in each soil zone (floating point, 1 column for each entry in landuse_zone_names) [-]*

zr is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "ls", "fao_process" will be set to "disabled". The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)* Additional columns: *Rooting depth in m for each land use zone (floating point, 1 column for each entry in landuse_zone_names) [L]*

kc is an *optional* input, provided as a path to either a YAML or CSV file. If not specified, "fao_process" will be set to "disabled". The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Crop coefficient for each land use zone (floating point, 1 column for each entry in landuse_zone_names) [-]*

taw is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "l", "fao_input" will be set to "ls". The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Total Available soil Water in mm for each land use zone (floating point, 1 column for each entry in landuse_zone_names) [-]*

raw is an *optional* input, provided as a path to either a YAML or CSV file. If not specified and "fao_input" is "l", "fao_input" will be set to "ls". The file should comprise multiple columns, as follows:

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Readily Available soil Water in mm for each land use zone (floating point, 1 column for each entry in landuse_zone_names) [-]*

percolation_rejection is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise a single column, as follows:

Single column: *percolation_rejection* keyword, followed by percolation limit in mm for each land use zone (floating point, 1 column for each entry in *landuse_zone_names*) [L]

percolation_rejection_use_timeseries is an *optional* input. It can be set to either true or false to control whether percolation rejection is assigned as a time series.

percolation_rejection_ts is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *percolation_rejection_ts*: keyword

1 additional row for each day (or user-defined time unit) in *time_periods*, followed by percolation limit in mm for each land use zone (floating point, 1 column for each entry in *landuse_zone_names*) [L]

Block: *Process*: Leakage below the root zone

These user inputs define parameters that govern the subroot leakage process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

subroot_leakage_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the leakage process is active in the model.

subroot_leakage_fraction is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (integer, number of entries = *num_nodes*)

Column 2: Proportion of subroot leakage permitted in each cell, linked to maximum rate listed in *subroot_leakage_ts* (floating point, number of entries = *num_nodes*) [-]

Block: *Process*: interflow

These user inputs define parameters that govern the interflow process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

interflow_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the interflow process is active in the model.

interflow_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: SWAc node number (integer, number of entries = *num_nodes*)

Column 2: *Zone number (integer, number of entries = num_nodes)*

interflow_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in interflow_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in interflow_zone_mapping)*

init_interflow_store is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in interflow_zone_mapping)*

Column 2: *Initial value for interflow store in mm (floating point, number of entries = max zone number in interflow_zone_mapping) [L]*

interflow_store_bypass is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in interflow_zone_mapping)*

Column 2: *Proportion of water from root zone percolation and subrootzone leakage bypassing the interflow store (floating point, number of entries = max zone number in interflow_zone_mapping) [L]*

infiltration_limit is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in interflow_zone_mapping)*

Column 2: *Maximum infiltration to groundwater from interflow store in mm (floating point, number of entries = max zone number in interflow_zone_mapping) [L]*

interflow_decay is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in interflow_zone_mapping)*

Column 2: *Proportion of interflow store in each day (or user-defined time unit) released to surface water (floating point, number of entries = max zone number in interflow_zone_mapping) [-]*

infiltration_limit_use_timeseries is an *optional* input. It can be set to either true or false to control whether the infiltration limit is assigned as a time series.

interflow_decay_use_timeseries is an *optional* input. It can be set to either true or false to control whether the interflow decay is assigned as a time series.

infiltration_limit_ts is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *infiltration_limit_ts*: keyword

1 additional row for each day (or user-defined time unit) in *time_periods*, followed by infiltration limit in mm for each interflow zone (floating point, 1 column for each entry in *interflow_zone_names*) [L]

interflow_decay_ts is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *interflow_decay_ts*: keyword

1 additional row for each day (or user-defined time unit) in *time_periods*, followed by interflow decay proportion for each interflow zone (floating point, 1 column for each entry in *interflow_zone_names*) [-]

Block: *Process*: Recharge release to saturated zone

These user inputs define parameters that govern the recharge attenuation process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

recharge_attenuation_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the recharge attenuation process is active in the model.

recharge_attenuation_params is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 4 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Initial recharge store in mm (floating point, number of entries = num_nodes) [L]*

Column 3: *Proportion of recharge store released each day (or user-defined time unit) (floating point, number of entries = num_nodes) [-]*

Column 4: *Maximum rate of release in mm (floating point, number of entries = num_nodes) [L]*

Block: *Process*: Solute transport

These user inputs define parameters that govern the solute transport process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

historical_solute_process is a *required* input. It can be set to either "enabled" or "disabled". If enabled, solute process must also be enabled and its outputs will be made available to a sequential (e.g. future-facing) run.

historical_mi_array_kg_per_time_period is an *optional* input (required only if **historical_solute_process** is enabled), provided as a path to a CSV file. The file is generally produced

by SWAc when simulating preceding solute transport for preceding time periods, thus giving starting conditions for subsequent time periods. It comprises a 2D matrix of floating-point values (in kg) and should be structured as follows:

1 row for every model cell [M]

1 column for every time period in the preceding run [M]

historical_start_date is an *optional* input (required only if `historical_solute_process` is enabled), provided in the format: "YYYY-MM-DD". It specifies the starting date of the preceding simulation and is used by SWAc to track remnant transport in the unsaturated zone during the current simulation period.

historical_time_periods is an *optional* input (required only if `historical_solute_process` is enabled), provided as a path to a CSV file. It defines the periods over which output data were aggregated in the preceding model, as defined by that model's `time_periods` keyword. The file should comprise 2 columns, as follows:

Column 1: *first day of this time period (integer, number of entries = number of time periods) [T]*

Column 2: *first day of next time period (i.e. end of current time period plus 1, notably in the final time period of the model) (integer, number of entries = number of time periods) [T]*

The normal unit of time is days, although seconds, hours or other can be entered if used as the standard unit of time in all files.

solute_process is a *required* input. It can be set to either "enabled" or "disabled". If enabled, entries are required for all subsequent parameters.

solute_calibration_a is an *optional* input (required only if `solute_process` is enabled), entered as a dimensionless floating-point value, representing the logarithm base in the following equation (described in Section 6.12 of the report):

$$F\left(\frac{1}{t}\right) = \frac{1}{2} \left[1 + \operatorname{erf} \left(-\frac{\log_a \left(a n_e DTW \frac{1}{t} \right) - \mu}{\sigma \sqrt{2}} \right) \right]$$

solute_calibration_mu is an *optional* input (required only if `solute_process` is enabled), provided as a path to a CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *fitting parameter mu in above equation (floating point, number of entries = num_nodes) [L]*

solute_calibration_sigma is an *optional* input (required only if `solute_process` is enabled), entered as a dimensionless floating-point value, fitting parameter sigma in the above equation.

solute_calibration_alpha is an *optional* input (required only if `solute_process` is enabled), entered as a dimensionless floating-point value, fitting parameter alpha in the above equation.

solute_calibration_effective_porosity is an *optional* input (required only if `solute_process` is enabled), provided as a path to a CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *effective porosity [-]*

solute_depth_to_water is an *optional* input (required only if **solute_process** is enabled), provided as a path to a CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *depth to water in m (floating point, number of entries = num_nodes) [L]*

solute_loading is an *optional* input (required only if **solute_process** is enabled), provided as a path to a CSV file. The file should comprise 11 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *UNIQUE (not used in SWAcMod)*

Column 3: *X (not used in SWAcMod)*

Column 4: *Y (not used in SWAcMod)*

Column 5: *Max load (kg/hectare)*

Column 6: *HER at 5% (mm/day)*

Column 7: *HER at 50% (mm/day)*

Column 8: *HER at 95% (mm/day)*

Column 9: *5PercLoadM (not used in SWAcMod)*

Column 10: *50PercLoad (not used in SWAcMod)*

Column 11: *95PercLoad (not used in SWAcMod)*

Block: *Process: SW release*

These user inputs define parameters that govern the surface water release process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

sw_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the SW release process is active in the model.

sw_params is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 3 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Initial SW store in mm (floating point, number of entries = num_nodes) [L]*

Column 3: *Proportion of SW store released each day (or user-defined time unit) (floating point, number of entries = num_nodes) [-]*

Block: *Process: SW ponding*

These user inputs define parameters that govern the surface water ponding process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

sw_ponding_process is a required input. It can be set to either "enabled" or "disabled" to control whether the SW release process is active in the model. *If enabled all FAO inputs should reflect open water.*

sw_zone_mapping is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = num_nodes)*

Column 2: *Zone number (integer, number of entries = num_nodes)*

sw_zone_names is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *Zone number (integer, number of entries = max zone number in sw_zone_mapping)*

Column 2: *Zone name (text string, enclosed by ' ', number of entries = max zone number in sw_zone_mapping)*

sw_downstream is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *sw_downstream: keyword*

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Proportion of ponded water released in each time unit (floating point, 1 column for each entry in sw_zone_names) [-]*

sw_activation is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows: Row 1: *sw_activation: keyword*

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Minimum water depth in mm before bed infiltration can occur (floating point, 1 column for each entry in sw_zone_names) [L]*

sw_bed_infiltration is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *sw_bed_infiltration: keyword*

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Proportion of ponded water infiltrating through bed to recharge attenuation store (floating point, 1 column for each entry in sw_zone_names) [-]*

sw_direct_recharge is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *sw_direct_recharge: keyword*

Column 1: *Month number (integer, number of entries = 12)*

Additional columns: *Proportion of ponded water leaving bed and bypassing recharge attenuation store (floating point, 1 column for each entry in sw_zone_names) [-]*

sw_pe_to_open_water is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *sw_pe_to_open_water: keyword*

Column 1: *Month number (integer, number of entries = 12)* Additional columns: *Open water evaporation coefficient for each zone (floating point, 1 column for each entry in sw_zone_names) [-]*

sw_init_ponding is a *required* input. *Initial ponding depth [mm]*

sw_max_ponding is a *required* input. *Max ponding depth [mm]*

sw_ponding_area is an *optional* input, provided as a path to either a YAML or CSV file. The file should be structured as follows:

Row 1: *sw_ponding_area:: keyword* Column 1: *Zone number (integer, number of entries = max zone number in sw_zone_mapping)*

Column 2: *Proportion of cell subject to ponding (floating point, 1 column for each entry in sw_zone_names) [-]*

Block: *Process: Surface Water Routing*

These user inputs define parameters that govern the surface water routing process. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

routing_process is a *required* input. It can be set to either "enabled" or "disabled" to control whether the routing process is active in the model.

routing_topology is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 11 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = number of nodes subject to EVT)*

Column 2: *downstr - swac node downstream of this one*

Column 3: *str_flag - 1 = stream cell, 0 = not stream cell*

Column 4: *node_mf - modflow node number of this node*

Column 5: *RCHLEN - the modflow SFR variable RCHLEN*

Column 6: *ca - the upstream contributing area*

Column 7: *STRTOP - the modflow SFR variable STRTOP*

Column 8: *STRTHICK* - the modflow SFR variable *STRTHICK*

Column 9: *STRHC1* - the modflow SFR variable *STRHC1*

Column 10: *DEPTH2* - the modflow SFR variable *DEPTH2*

Column 11: *WIDTH2* - the modflow SFR variable *WIDTH2*

output_sfr is an *optional* input, provided as a true or false statement. *If not specified, it will be set to "false."*

attenuate_sfr_flows is an *optional* input, provided as a true or false statement. *This parameter only applies when output_sfr is true.*

sfr_obs is an *optional* input, provided as a Modflow 6 sfr obs input file.

istcb1 is an *optional* input, provided as an integer. *If not specified, it will be set to 50.*

istcb2 is an *optional* input, provided as an integer. *If not specified, it will be set to 55.*

swdis_ts is an *optional* input, provided as a path to either a YAML or CSV file. One column for each SW discharge location in *swdis_locs*. SW discharge rate in m³/day (or other user-defined time unit) (floating point, number of rows = maximum value in *time_periods*).

swdis_locs is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = number of nodes with SW discharges)*

Column 2: *Index number of discharge from 1 to number of nodes with SW discharges (integer, number of entries = number of nodes with SW discharges)*

swabs_ts is an *optional* input, provided as a path to either a YAML or CSV file. One column for each SW abstraction location in *swabs_locs*. SW abstraction rate in m³/day (or other user-defined time unit) (floating point, number of rows = maximum value in *time_periods*).

swabs_locs is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 2 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = number of nodes with SW abstractions)*

Column 2: *Index number of abstraction from 1 to number of nodes with SW abstractions (integer, number of entries = number of nodes with SW abstractions)*

Block: Time Series

These user inputs specify the time series needed for the main model processes, as well as the MODFLOW model type that the SWAc output should follow. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

rainfall_ts is a *required* input, provided as a path to either a YAML or CSV file. One column for each entry in rainfall_zone_names. Rainfall in mm (floating point, number of rows = maximum value in time_periods) [L].

pe_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the FAO or the Canopy process is enabled. One column for each entry in pe_zone_names. Rainfall in mm (floating point, number of rows = maximum value in time_periods) [L].

temperature_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the Snow process is enabled. One column for each entry in temperature_zone_names. Temperature in °C (floating point, number of rows = maximum value in time_periods) [C].

tmax_c_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the Snow process is enabled. One column for each entry in tmax_c_zone_names. Max temperature in C (floating point, number of rows = maximum value in time_periods) [C].

tmin_c_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the Snow process is enabled. One column for each entry in tmin_c_zone_names. Min temperature in C (floating point, number of rows = maximum value in time_periods) [C].

windsp_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the Snow process is enabled. One column for each entry in windsp_zone_names. Wind speed in m/s (floating point, number of rows = maximum value in time_periods) [L/T].

subroot_leakage_ts is an *optional* input, provided as a path to either a YAML or CSV file. This parameter is required only if the Leakage process is enabled. One column for each entry in subroot_zone_names. Maximum subroot leakage rate in mm (floating point, number of rows = maximum value in time_periods) [L].

gwmodel_type is a *required* input, provided as a single keyword: mf6, mf96 or mfulg.

mf96_lrc is an *optional* input, provided as a list of three integers. This parameter is only read if mf96 is entered for gwmodel_type. The list represents the number of layers, rows and columns in the model.

Block: EVT

These user inputs specify the handling of unutilised Potential Evapotranspiration (PE), after all the main model processes have been calculated. As with all subsequent blocks, it comprises a series of pre-defined keywords in the master control file (see Appendix A), followed by either a mandatory or optional user response, either as a file path, or as a direct entry in the master control file. The required format for each keyword is listed below.

Explanation of keywords

output_evt is an *optional* input, provided as a boolean (true or false). If not specified, it will be set to "false".

excess_sw_process is a *required* input, provided as a string. The only valid value is "disabled".

evt_parameters is an *optional* input, provided as a path to either a YAML or CSV file. The file should comprise 4 columns, as follows:

Column 1: *SWAc node number (integer, number of entries = number of nodes subject to EVT)*

Column 2: *MF node number (integer, number of entries = number of nodes subject to EVT)*

Column 3: *Elevation of the ET surface in m (floating point, number of entries = number of nodes subject to EVT) [L]*

Column 4: *ET extinction depth in m (floating point, number of entries = number of nodes subject to EVT) [L]*

ievtcb is an *optional* input, provided as an integer. If not specified, it will be set to 50.

nevtopt is an *optional* input, provided as an integer. If not specified, it will be set to 2.